

神经网络量化实验

陈天乐 无 25

2025 年 4 月 1 日

目录

1	实验目的	2
2	基本任务	2
2.1	可视化权重分布	2
2.2	量化网络权重	4
2.3	量化激活层	4
2.4	量化 bias	11
2.5	对比 QAT 和 PTQ 两种方法	12
2.6	其他探索——更换网络结构	14
3	实验总结	16
4	实验源代码	17

1 实验目的

通过对一个简单的卷积神经网络的量化，掌握对神经网络量化的基本方法，更加深入地感受量化对神经网络训练和推理的影响，并对比 PTQ 和 QAT 两种训练方法的联系与区别。

2 基本任务

2.1 可视化权重分布

1. 工程中已给出了如何得到当前网络参数的直方图、最大最小值、以及 3-sigma 范围的代码。记录这些数据，你有什么发现？

记录的数据和直方图如下所示：

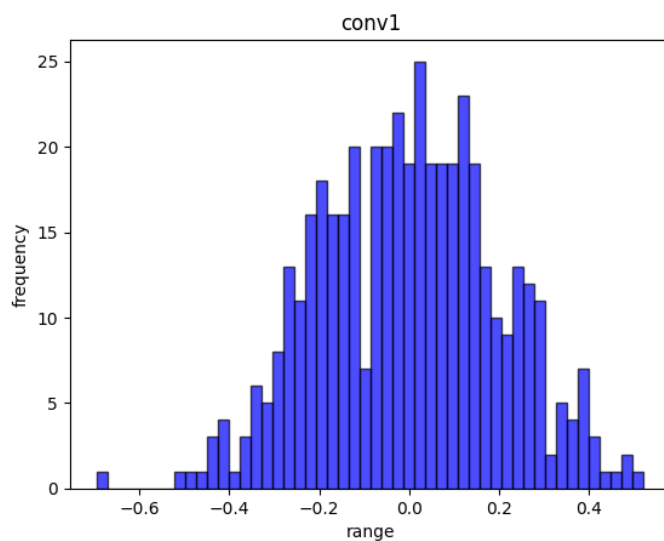


图 1: conv1 的权重分布直方图

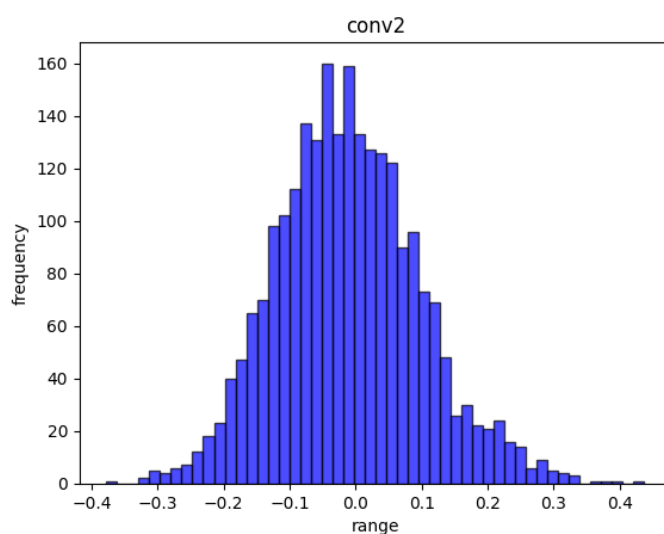


图 2: conv2 的权重分布直方图

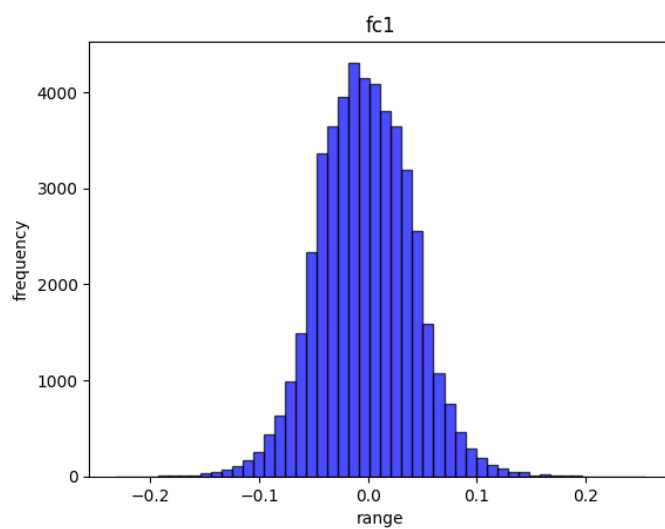


图 3: fc1 的权重分布直方图

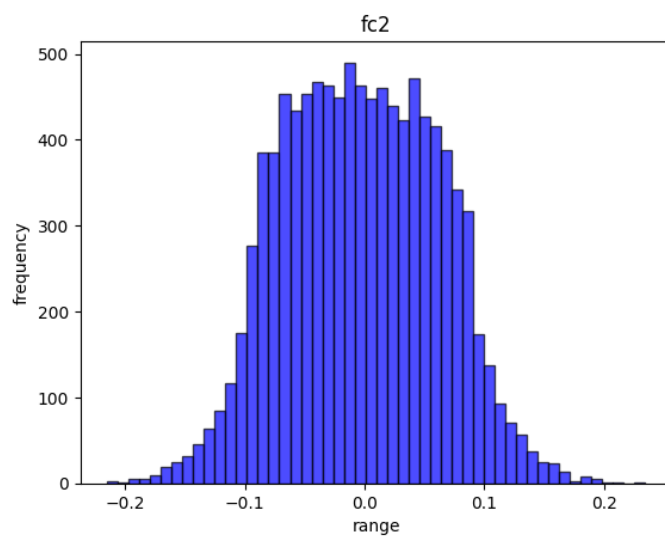


图 4: fc2 的权重分布直方图

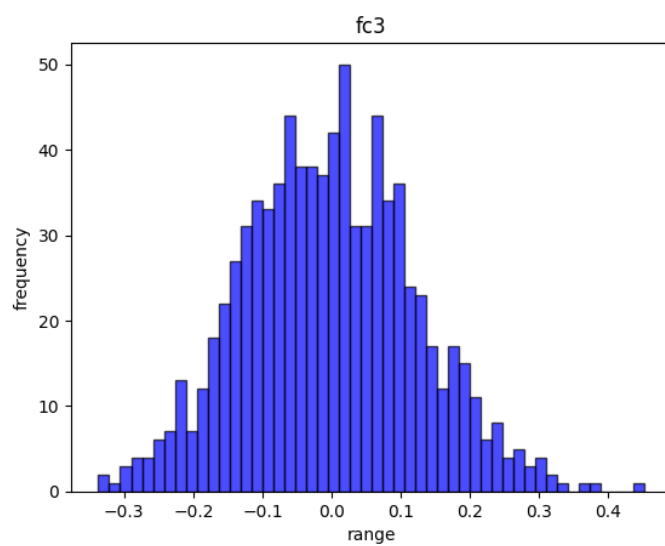


图 5: fc3 的权重分布直方图

层	最大值范围	3-sigma 范围
conv1	[-0.69379, 0.52103]	[-0.69379, 0.52103]
conv2	[-0.37755, 0.43689]	[-0.33182, 0.33182]
fc1	[-0.23190, 0.25502]	[-0.14487, 0.14487]
fc2	[-0.21480, 0.23412]	[-0.18090, 0.18090]
fc3	[-0.33725, 0.45266]	[-0.33725, 0.37426]

2. 你会更倾向于选择绝对值最大值作为量化 scale，还是 3-sigma range，为什么？

观察分布直方图和记录的结果之后，我更倾向于选择 3-sigma 范围作为量化 scale。有以下几个原因：

1. 首先，3-sigma 范围中已经包含了 99.73% 的数据，而这个范围会把一些占极少数边缘值的影响给去除掉，同时又能包含绝大多数数据的量化准确性
2. 其次，3-sigma 范围相较于最大值范围更加对称，同时范围更小，也就意味着量化时不同值之间的间隔会更大，我认为这样可以降低量化误差对模型准确度的影响

2.2 量化网络权重

1. 工程文件中已经给出了对网络权重参数的量化代码，工程里使用的是 3-sigma range，你可以自己尝试更换其他 range，来看看效果。（不考虑 bias）

使用 3-sigma range 量化后，模型推理的准确率为 54.29%（完全不量化时模型准确率是 54.4%）；使用最大值范围量化后，模型推理的准确率为 51.1%，发生了明显的下降，因此还是 3-sigma 范围的效果更好。

2. 记录仅量化权重后的模型精度。当我们把 float32 的权重量化到 8bit 整数时，发生了什么变化？网络参数乘以 scaling factor 后，参数的绝对大小发生了很大变化，为什么我们可以不管 weight 的绝对大小，只需要保留相对的大小就行了？这样操作是不是对所有任务都适用呢？

仅量化权重后，模型推理精度为 54.29%，可以发现，把 float32 的权重量化到 8bit 整数时，除了模型的绝对大小发生了变化，其权重的相对大小关系是不变的，模型的准确率也没有明显的变化。我们之所以可以只关注 weight 的绝对大小的原因是因为，目前的网络都会有 BatchNorm 层做处理，同时使用的激活函数 ReLU 也可以自适应调整，因此绝对大小不会影响其功能。但并不是所有任务都适用，如果是高精度的回归任务，对数值精度异常敏感，又或者是无归一层的模型，或是低比特量化（1, 2, 4bit），那么这样的操作很有可能会让模型 breakdown。

2.3 量化激活层

1. 请仿照 1.1 的代码，记录激活层直方图，最大最小值，3-sigma 范围的代码。

补全的代码如下：

```

1  ## Question 3.1:
2
3  # Plot histograms of the input images and the outputs of every convolutional and fully-
   connected layer.
4  # Record any observations you make about the distribution of the values.

```

```

5 # ADD YOUR CODE HERE
6 Activation_border = 2*(bitwidth_A-1)
7 input_activations = net_q3.input_activations # 输入数据
8 conv1_output_activations = net_q3.conv1.activations # 第一层卷积层的输出数据
9 conv2_output_activations = net_q3.conv2.activations # 第二层卷积层的输出数据
10 fc1_output_activations = net_q3.fc1.activations # 同上类推
11 fc2_output_activations = net_q3.fc2.activations
12 fc3_output_activations = net_q3.fc3.activations
13
14
15 # to plot distributions of activations
16 # Plot histograms of the following variables, and calculate their ranges and 3-sigma
    ranges:
17 # 拼接上面的数据成一个group, 方便循环遍历
18 activation_groups = [
19     (input_activations, "input_activations"),
20     (conv1_output_activations, "conv1_output_activations"),
21     (conv2_output_activations, "conv2_output_activations"),
22     (fc1_output_activations, "fc1_output_activations"),
23     (fc2_output_activations, "fc2_output_activations"),
24     (fc3_output_activations, "fc3_output_activations"),
25 ]
26
27 for activation_data, name in activation_groups: # 遍历类的成员
28     print(name + ": the range is [{:.5f}, {:.5f}]" .format(activation_data.min(),
        activation_data.max()))
29
30     # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
31     sigma3 = int(np.size(activation_data)*0.0027+0.5)
32     sigma3_value = np.abs(activation_data)[np.argsort(np.abs(activation_data), -
        sigma3)[-sigma3]]
33     print(name + ": the 3-sigma range is [{:.5f}, {:.5f}]" .format(max(-sigma3_value,
        activation_data.min()), min(sigma3_value, activation_data.max())))
34
35     plt.hist(activation_data, bins=50, facecolor="green", edgecolor="black", alpha=0.7)
36     # 画直方图
37     plt.xlabel("range")
38     plt.ylabel("frequency")
39     plt.title(name+"_distribution")
40     plt.show()
41
42 # You can get a flattened vector of the weights of fc1 like this:
43 # fc1_weights = net.fc1.weight.data.cpu().view(-1)

```

Listing 1: lab1_PTQ.ipynb

记录的数据和直方图如下所示:

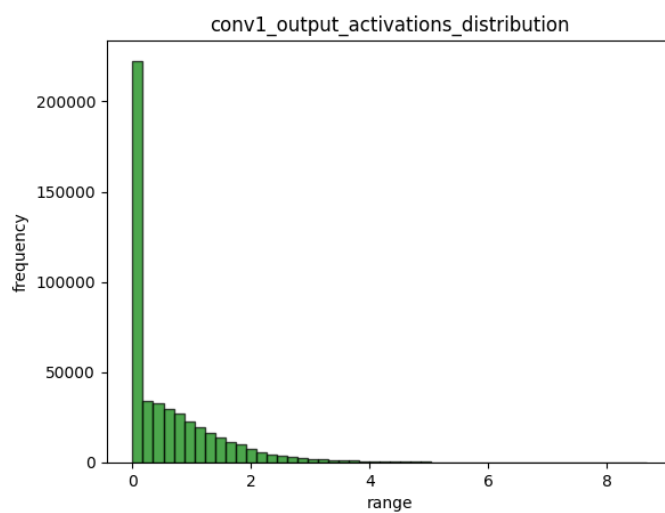


图 6: conv1 的激活值分布直方图

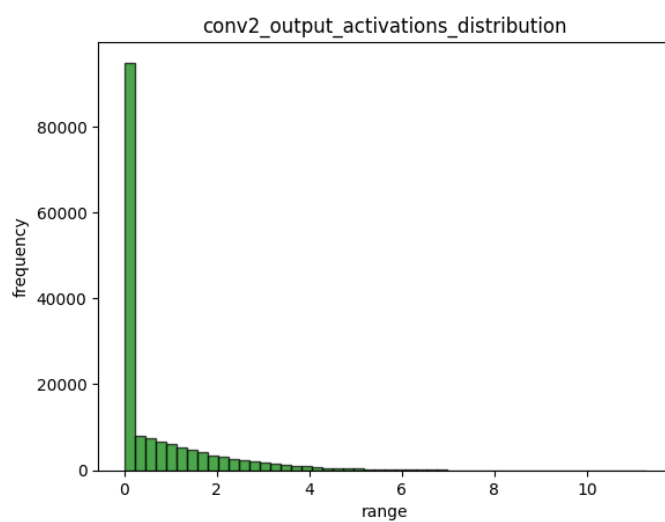


图 7: conv2 的激活值分布直方图

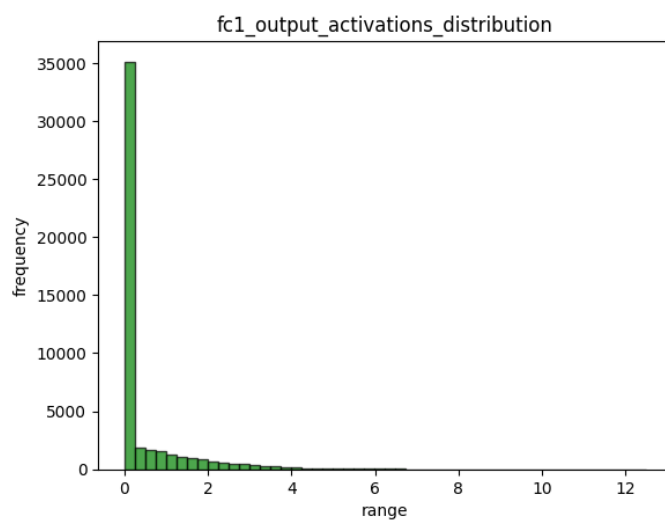


图 8: fc1 的激活值分布直方图

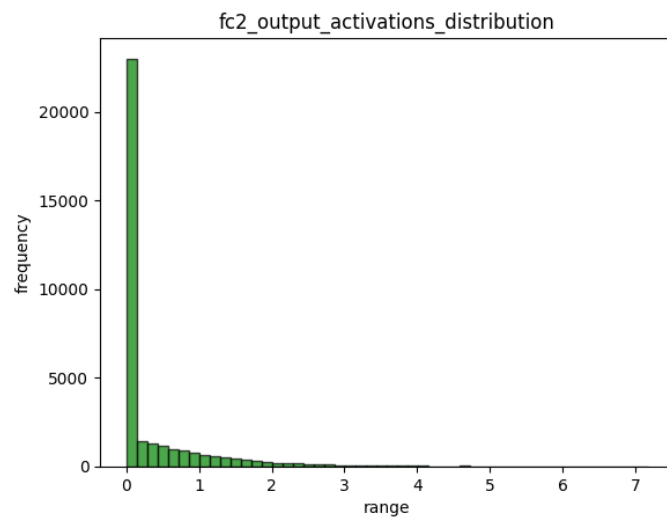


图 9: fc2 的激活值分布直方图

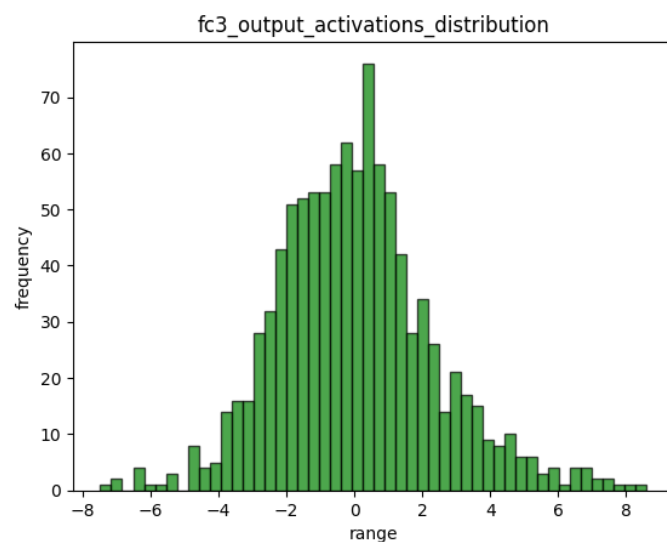


图 10: fc3 的激活值分布直方图

层	最大值范围	3-sigma 范围
input	[-1.00000, 1.00000]	[-1.00000, 1.00000]
conv1	[0.00000, 8.67846]	[0.00000, 4.56645]
conv2	[0.00000, 11.26935]	[0.00000, 6.56817]
fc1	[0.00000, 12.47395]	[0.00000, 6.19761]
fc2	[0.00000, 7.16761]	[0.00000, 4.04248]
fc3	[-7.48732, 8.60905]	[-7.48732, 7.84135]

2. 你会更倾向于选择绝对值最大值作为 scaling factor，还是 3-sigma range，为什么？

和 2.1 部分的阐述理由一致，并且激活值的最大值范围和 3-sigma 值差距更大了，使用 3-sigma range 的效果和使用最大值范围的差距可能会比量化权重时要更大更明显。

3. 请仿照 2.1 的代码，对激活层进行量化。提示：本层输出的 scaling factor 与浮点数表示的

本层输出范围、输入的 scaling factor 和 weight 的 scaling factor 都相关，可以参考课件第 21 页或者附件中的论文 2.1-2.2 章。

补全的代码如下：

```

1 from typing import List
2
3 class NetQuantized(nn.Module):
4     def __init__(self, net_with_weights_quantized: nn.Module, bitwidth_A):
5         super(NetQuantized, self).__init__()
6
7         net_init = copy_model(net_with_weights_quantized) # 最初的网络copy 过来,用于初始
            量化网络
8
9         self.conv1 = net_init.conv1
10        self.pool = net_init.pool
11        self.conv2 = net_init.conv2
12        self.fc1 = net_init.fc1
13        self.fc2 = net_init.fc2
14        self.fc3 = net_init.fc3
15        self.activation_border = 2*(bitwidth_A-1)
16        print(self.activation_border)
17
18        for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
19            #逐层遍历,检查量化是否成功
20            def pre_hook(l, x):
21                x = x[0]
22                #异常处理:检查之前的量化是否将所有的数转化进规定的区间
23                if (x < -self.activation_border).any() or (x > (self.activation_border
24                    -1)).any():
25                    raise Exception("Input to {} layer is out of bounds for an {}-bit
26                        signed integer".format(l.__class__.__name__, bitwidth_A))
27                    if (x != x.round()).any():
28                        raise Exception("Input to {} layer has non-integer values".format(l.
29                            __class__.__name__))
30
31                layer.register_forward_pre_hook(pre_hook)
32
33            # Calculate the scaling factor for the initial input to the CNN
34            self.input_activations = net_with_weights_quantized.input_activations
35            self.input_scale = NetQuantized.quantize_initial_input(self.input_activations,
36                self.activation_border)
37            #以上2行计算了input activation 的scale和对应的quantization 版本
38            # Calculate the output scaling factors for all the layers of the CNN
39
40            n_w = []
41            activation_scale= [self.input_scale]
42
43            for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
44                # call quantize_activations and add your code here
45
46                #获取权重缩放因子
47                n_w.append(layer.weight.scale)
48
49                # 计算当前层激活的缩放因子
50                activation_scale.append( NetQuantized.quantize_initial_input(
51                    layer.activations, 2*self.activation_border
52                ) )
53
54            #print(activation_scale)
55
56            n = 0

```



```

53         for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
54             layer.output_scale = NetQuantized.quantize_activations(n_w[n],
55                             activation_scale[n], activation_scale[n + 1])
56             n = n + 1
57
58         # 注册前向检查钩子
59         for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
60             def pre_hook(l, x):
61                 x = x[0]
62                 if (x < -self.activation_border).any() or (x > (self.activation_border
63                     -1)).any():
64                     raise Exception(f"Input to {l.__class__.__name__} layer is out of
65                         bounds")
66                 if (x != x.round()).any():
67                     raise Exception(f"Input to {l.__class__.__name__} layer has non-
68                         integer values")
69                 layer.register_forward_pre_hook(pre_hook)
70
71         # 计算输入的scale函数
72         @staticmethod
73         def quantize_initial_input(pixels: np.ndarray, activation_border:int):
74             '''
75             Calculate a scaling factor for the images that are input to the first layer of
76             the CNN.
77
78             Parameters:
79             pixels (ndarray): The values of all the pixels which were part of the input
80             image during training
81
82             Returns:
83             float: A scaling factor that the input should be multiplied by before being fed
84             into the first layer.
85             This value does not need to be an 8-bit integer.
86             '''
87
88             sigma3 = int(np.size(pixels)*0.0027+0.5)
89             sigma3_value = pixels[np.argpartition(pixels,-sigma3)[-sigma3]]
90             max_value = np.max(np.abs(pixels))
91             scale = activation_border/max_value # you can modify the range here, 同样, 这个
92             地方的scale和论文定义的是和源代码倒数的关系
93             return scale
94
95         # 逐层激活送入量化
96         @staticmethod
97         def quantize_activations( n_w: float, pre_activation_scale: float,
98                                 now_activation_scale: float) -> float:
99             '''
100             Calculate a scaling factor to multiply the output of a layer by.
101
102             Parameters:
103             activations (ndarray): The values of all the pixels which have been output by
104             this layer during training
105             n_w (float): The scale by which the weights of this layer were multiplied as
106             part of the "quantize_weights" function you wrote earlier
107             n_initial_input (float): The scale by which the initial input to the neural
108             network was multiplied
109             ns ([[float, float]]): A list of tuples, where each tuple represents the "weight
110             scale" and "output scale" (in that order) for every preceding layer
111             #用一个元组记录已经算好的所有网络的weight scale 和 output scale
112             Returns:
113             float: A scaling factor that the layer output should be multiplied by before

```

```

being fed into the first layer.
    This value does not need to be an 8-bit integer.
'''

# ADD YOUR CODE HERE
scale = now_activation_scale / (pre_activation_scale * n_w)
return scale

def forward(self, x: torch.Tensor) -> torch.Tensor:
    # You can access the output activation scales like this:
    # fc1_output_scale = self.fc1.output_scale

    # 这里其实是对定点数计算的模拟仿真，因此每层的输出均需要手动乘以scaling factor，
    并限制在规定的表示范围内。
    # output_scale:不同网络层之间传递整数重新压缩为8bit的scale(在第三周阅读论文中是
    定点浮点数M)
    # activation_scale:网络层内数据和真实值差距的scale
    # weight_scale:权重量化值和真实值差距的scale
    x = x*self.input_scale
    x = torch.clamp(x.round(), min=-self.activation_border, max=self.
activation_border-1) # 8bit是-128~127

    x = self.pool(F.relu(self.conv1(x)))
    x = x*self.conv1.output_scale
    x = torch.clamp(x.round(), min=-self.activation_border, max=self.
activation_border-1)

    x = self.pool(F.relu(self.conv2(x)))
    x = x*self.conv2.output_scale
    x = torch.clamp(x.round(), min=-self.activation_border, max=self.
activation_border-1)

    x = x.view(-1, 16 * 5 * 5)
    x = F.relu(self.fc1(x))
    x = x*self.fc1.output_scale
    x = torch.clamp(x.round(), min=-self.activation_border, max=self.
activation_border-1)

    x = F.relu(self.fc2(x))
    x = x*self.fc2.output_scale
    x = torch.clamp(x.round(), min=-self.activation_border, max=self.
activation_border-1)

    x = self.fc3(x)*self.fc3.output_scale
    return x

```

Listing 2: lab1_PTQ.ipynb

得到的模型推理准确率为:

Accuracy of the network after quantizing both weights and activations: 54.24%

图 11: 激活值和权重均量化后模型的推理准确率

量化后模型精度和未量化时几乎没有差别，说明了量化方法实现的正确性。

2.4 量化 bias

在未量化时，模型推理精度为：

Accuracy of the network (with a bias) on the test images: 54.11%

图 12: 带有偏置的模型推理准确率

将网络的权重量化但是偏置未量化后，模型的推理精度为：

Accuracy of the network on the test images after all the weights are quantized but the bias isn't: 51.4%

图 13: 带有偏置且量化权重的模型推理准确率

补全量化偏置的代码如下：

```

1 # slightly clearer bias bounds (32b signed integer)
2 #bias 量化后的最大值、最小值
3 MIN_32B_SINT = -(2**31)
4 MAX_32B_SINT = (2**31) - 1
5
6 class NetQuantizedWithBias(NetQuantized):
7     def __init__(self, net_with_weights_quantized: nn.Module):
8         super(NetQuantizedWithBias, self).__init__(net_with_weights_quantized, bitwidth_A)
9
10        preceding_scales = [(layer.weight.scale, layer.output_scale) for layer in self.
11        children() if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear)][:-1]
12        #print(preceding_scales)
13
14        self.fc3.bias.data = NetQuantizedWithBias.quantized_bias(
15            self.fc3.bias.data,
16            self.fc3.weight.scale,
17            self.input_scale,
18            preceding_scales
19        )
20
21        if (self.fc3.bias.data < MIN_32B_SINT).any() or (self.fc3.bias.data > MAX_32B_SINT).
22        any():
23            raise Exception("Bias has values which are out of bounds for an 32-bit signed
24            integer")
25            if (self.fc3.bias.data != self.fc3.bias.data.round()).any():
26                raise Exception("Bias has non-integer values")
27
28        @staticmethod
29        def quantized_bias(bias: torch.Tensor, n_w: float, n_initial_input: float, ns: List[Tuple[
30            float, float]]) -> torch.Tensor:
31            '''
32            Quantize the bias so that all values are integers between MIN_32B_SINT and
33            MAX_32B_SINT.
34
35            Parameters:
36            bias (Tensor): The floating point values of the bias
37            n_w (float): The scale by which the weights of this layer were multiplied
38            n_initial_input (float): The scale by which the initial input to the neural network
39            was multiplied
40            ns [(float, float)]: A list of tuples, where each tuple represents the "weight scale
41            " and "output scale" (in that order) for every preceding layer

```

```

36 0
37     Returns:
38     Tensor: The bias in quantized form, where every value is an integer between
MIN_32B_SINT and MAX_32B_SINT.
39         The "dtype" will still be "float", but the values themselves should all be
integers.
40         '''
41
42     # ADD YOUR CODE HERE
43
44     #print(bias)
45     # 计算累积缩放因子: 初始输入缩放 * 所有前驱层的输出缩放
46
47     scale = n_initial_input
48     for weight_scale, output_scale in ns:
49         scale *= output_scale
50         scale *= weight_scale
51
52     # 总缩放因子 = 累积输入缩放 * 当前层权重缩放
53     scale = scale * n_w
54     print(scale)
55
56     # 量化偏置: bias * scale → 四舍五入 → 截断至32位整数范围
57     quan_bias = (bias * scale).round()
58     quan_bias = torch.clamp(quan_bias, min=MIN_32B_SINT, max=MAX_32B_SINT)
59     return quan_bias
60
61 net_quantized_with_bias = NetQuantizedWithBias(net_with_bias_with_quantized_weights)
62
63 score = test(net_quantized_with_bias, testloader)

```

Listing 3: lab1_PTQ.ipynb

得到的模型精度为:

```
Accuracy of the network on the test images after all the weights and the bias are quantized: 52.84%
```

图 14: 量化偏置后的模型精度

模型精度甚至比只量化权重不量化偏置时的精度更高，也说明了量化偏置实现方法的正确性。

2.5 对比 QAT 和 PTQ 两种方法

PTQ 方法是在模型完成训练后直接对模型的权重、激活值和偏置做量化，而 QAT 是保持浮点数训练过程，但是会引入量化误差，同时修改了全连接层和卷积层的定义。首先在数据集上以 FP32 精度进行模型训练，得到训练好的 baseline 模型；然后在 baseline 模型中插入伪节点，对数据进行量化并立即反量化，从而引入了量化误差。部分实现的方法如下

```

1 def forward(self, input):
2     max_val = quant_max(input)
3     weight = input * self.qmax / max_val
4     q_weight = self.round(weight)
5     q_weight = q_weight * max_val / self.qmax # 通过这种方式引入量化误差，从而实现QAT
6     return q_weight
7
8 ...
9

```

```

10 def forward(self, x):
11     act = x
12     self._ema(x)
13     self.alpha.data = self.max_val.unsqueeze(0)
14     act = torch.max(torch.min(x, self.alpha), -self.alpha)
15
16     act = act * self.qmax / self.alpha
17     q_act = self.round(act)
18     q_act = q_act * self.alpha / self.qmax # 引入量化误差
19
20     return q_act

```

Listing 4: lab1_QAT.ipynb

但是观察代码可知，在 QAT 中定义了一个 `Torch.round()` 函数，这是因为在做反向传播时，梯度通过伪量化节点进行传递，模型参数的更新因此包含了量化误差的影响。但是按照正传的公式，由于量化后的权重是离散的，导致反向传播时：

$$\frac{\partial Q(W)}{\partial W} = 0$$

这样导致梯度无法传播，所以权重无法学习更新。因此采用 STE（直通估计器）将梯度通过量化传播，近似假设量化操作的梯度为 1，所以：

$$grad(W) = \frac{\partial L}{\partial W} = \frac{\partial L}{\partial Q(W)}$$

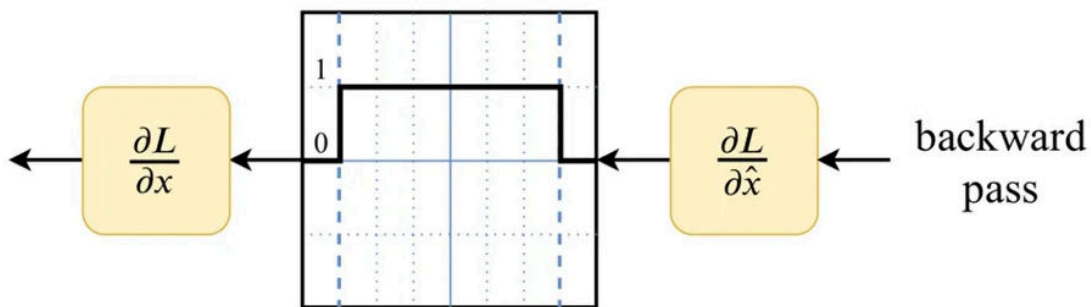


图 15: 反向传播模型

对于 PTQ 和 QAT 这两种方法的优缺点，列举如下：

1. PTQ

训练后量化的方式可以快速部署模型，直接在预训练模型上应用量化，节省时间和计算资源。其次就是硬件的兼容性好，主流推理框架对 PTQ 优化充分，支持多种硬件加速。但是由于是直接对训练完成后的模型进行量化，对复杂模型可能造成显著精度下降，尤其在低比特量化时非常明显。

2. QAT

量化感知训练仍然采用浮点数训练，并且在训练时引入量化误差，使模型参数适应量化后的数值分布，精度损失更小，鲁棒性也更高。但是需要重新训练或微调模型，耗费大量时间和计算资源，计算成本更高，部署的复杂度也更高。

2.6 其他探索——更换网络结构

首先，我在原先网络的基础上修改了一下，将 2 层卷积层增加到了 4 层，并且去除掉了一层全连接层，进行 10 个 epoch 的训练，最终模型的精度如下图和表格所示：

#	**Layer**	**Type**	**Input Shape**	**Output Shape**	**Activation**
#	conv1	Convolutional	3x32x32	16x32x32	ReLU
#	conv2	Convolutional	16x32x32	16x32x32	ReLU
#	pool1	Max pool	16x32x32	16x16x16	None
#	conv3	Convolutional	16x16x16	32x16x16	ReLU
#	conv4	Convolutional	32x16x16	32x16x16	ReLU
#	pool2	Max pool	32x16x16	32x8x8	None
#	fc1	Fully-connected	32x8x8	128	ReLU
#	fc2	Fully-connected	128	10	ReLU

图 16: 网络结构

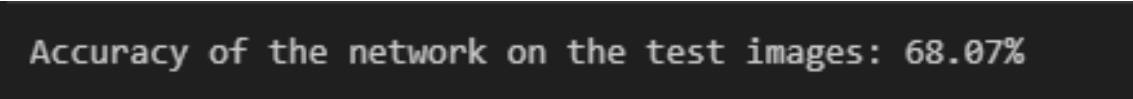


图 17: 未量化时的模型精度

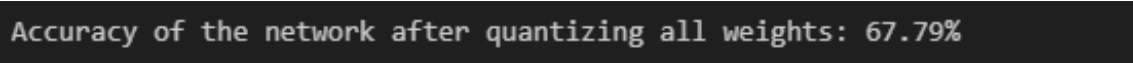


图 18: 量化权重时的模型精度

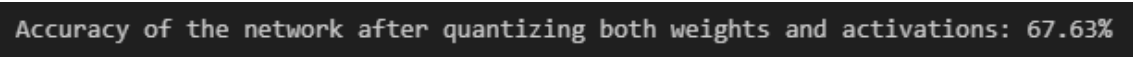


图 19: 量化权重和激活值时的模型精度

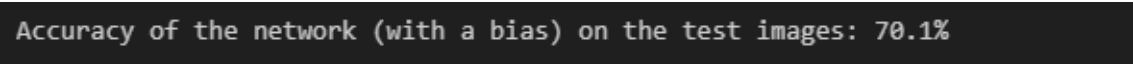


图 20: 带权重时的模型精度

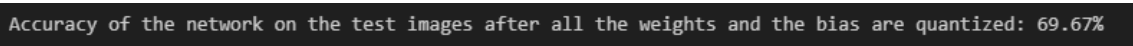


图 21: 量化权重时的模型精度

然后，我又将网络结构修改为了 ResNet，但是由于输入图片尺寸较小，ResNet 的深度只有 8 个卷积层。一开始在实现 ResNet 时没有加入 BatchNorm 层，导致模型训练速度很慢，

模型	模型精度
原始模型	68.07%
仅量化权重	67.79%
量化激活值和权重	67.73%
带偏置	70.01%
量化偏置	69.79%

同时在量化权重后网络的精度直接降低到了 14% 左右。后面将 BN 层补上后网络性能恢复，其具体结构和训练精度如下图和表格所示：

#	**Layer**	**Type**	**Input Shape**	**Output Shape**	**Activation**
#	conv1	Convolutional	3x32x32	64x32x32	ReLU
#	pool1	Max pool	64x32x32	64x16x16	None
#	conv2	Convolutional	64x16x16	64x16x16	ReLU
#	conv3	Convolutional	64x16x16	64x16x16	ReLU
#	conv4	Convolutional	64x16x16	64x16x16	ReLU
#	conv5	Convolutional	64x16x16	128x8x8	ReLU
#	conv6	Convolutional	128x8x8	128x8x8	ReLU
#	conv7	Convolutional	128x8x8	128x8x8	ReLU
#	conv8	Convolutional	64x16x16	128x8x8	ReLU
#	pool2	Avg pool	128x8x8	128x4x4	None
#	fc1	Fully-connected	128x4x4	128	ReLU
#	fc2	Fully-connected	128	10	ReLU

图 22: 网络结构

Accuracy of the network on the test images: 77.8%

图 23: 未量化时的模型精度

Accuracy of the network after quantizing all weights: 77.5%

图 24: 量化权重时的模型精度

Accuracy of the network after quantizing both weights and activations: 64.81%

图 25: 量化权重和激活值时的模型精度

Accuracy of the network (with a bias) on the test images: 74.48%

图 26: 带权重时的模型精度

Accuracy of the network on the test images after all the weights are quantized but the bias isn't: 74.62%

图 27: 量化权重时的模型精度

模型	模型精度
原始模型	77.8%
仅量化权重	77.5%
量化激活值和权重	64.81%
带偏置	74.48%
量化偏置	74.62%

此外，我还对 ResNet 网络进行了 QAT 训练，在这个训练中能明显感受到 QAT 的缺陷：训练时间极长，训练计算成本太高，不利于快速部署。大概训练 2 个 epoch 的时间可以让 PTQ 后的模型训练 10 个 epoch。

有关这一部分的代码都放在了源代码中，还麻烦助教查看。

3 实验总结

在本次实验中，通过对卷积神经网络的 PTQ、QAT 量化方案的实现，成功地将课上论文中提及的量化方法复现，并且对其做了很多作业要求之外的探索。除了我做的更换网络结构部分，小组成员也对更换激活函数、低比特量化等不同方向做了探索，对于量化这一部分的理解和掌握得到了很好的提高。也感谢在本次实验中辛苦付出的同组同学，和答疑解惑的助教、老师们！

4 实验源代码

```

1  # %%
2  # 本次实验包括后量化的过程，主要在于学习如何对DNN的权重（weight）和激活层（activation）进行量
   化。
3
4  # %%
5  # 引用外部软件包
6  import torch # pytorch包
7  import torchvision # torchvision是pytorch官方提供的工具，包含很多常用数据集的封装等
8  import torchvision.transforms as transforms
9  import torch.nn as nn # nn 和下面的F均包含了主要的神经网络模块，包括卷积层、全连接层、池化层等
   等
10 import torch.nn.functional as F
11 import torch.optim as optim # 优化器
12
13 # %%
14 # 确保device为gpu，否则会花费过长时间。
15 if torch.cuda.is_available():
16     torch.cuda.set_device(0) # 0为选择显卡id
17     device = "cuda"
18     print('using cuda')
19 else:
20     device = torch.device('cpu')
21     print('using cpu')
22
23 # 设置量化位宽
24 bitwidth_A = 8
25 bitwidth_W = 8
26
27 # %%
28 # Load training and test data from the CIFAR10 dataset.
29 # torchvision是pytorch官方集成的工具库，包括学界常用的数据集和数据预处理方式。
30 transform = transforms.Compose(
31     [transforms.ToTensor(), # 将数据归一化，并转换成tensor的形式
32      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]) # 预处理数据的方式，normalize是
   将图像进行规则化。
33
34 trainset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数字系
   统设计/lab1/dataset', train=True,
35                                         download=True, transform=transform) #定义了训练集，
   path是训练数据所在地址，指定位置自动下载。
36 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
37                                         shuffle=True, num_workers=2) #定义数据迭代器，
   batchsize为一次迭代训练，送进去图像的数量（可以尝试调整，看看会发生什么）。shuffle表示要
   不要打乱训练图像的排序。
38
39 testset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数字系
   统设计/lab1/dataset', train=False,
40                                         download=True, transform=transform)
41 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
42                                         shuffle=False, num_workers=2)
43
44 # %%
45 # Define a simple CNN that classifies CIFAR images.
46 # The network provided is similar to LeNet-5, and it has the following architecture:
47
48 # **Layer** |      **Type**      | **Input Shape** | **Output Shape** | **Activation**
49 #  conv1    | Convolutional      | 3x32x32         | 6x28x28          | ReLU
50 #  pool1    | Max pool           | 6x28x28         | 6x14x14          | None
51 #  conv2    | Convolutional      | 6x14x14         | 16x10x10         | ReLU

```

```

52 # pool2 | Max pool | 16x10x10 | 16x5x5 | None
53 # fc1 | Fully-connected | 400 | 120 | ReLU
54 # fc2 | Fully-connected | 120 | 84 | ReLU
55 # fc3 | Fully-connected | 84 | 10 | None
56
57 # None of the layers in the network have a bias associated with them.
58 # This makes them easier to quantize.
59 # Towards the end of this assignment, we will add biases to the final layer and quantize it as
    well.
60
61 # %%
62 # 网络定义，conv2d是卷积层，linear是全连接层。forward下定义了计算图，显示送进来的图像（x）会怎
    么处理。
63 class Net(nn.Module):
64     # 网络结构：2层卷积层、1层最大值池化层、3层全连接层，留意这些参数的内涵
65     def __init__(self):
66         super(Net, self).__init__()
67         self.conv1 = nn.Conv2d(3, 6, 5, bias=False)
68         self.pool = nn.MaxPool2d(2, 2) # run after each conv (hence the 5x5 FC layer)
69         self.conv2 = nn.Conv2d(6, 16, 5, bias=False)
70         self.fc1 = nn.Linear(16 * 5 * 5, 120, bias=False)
71         self.fc2 = nn.Linear(120, 84, bias=False)
72         self.fc3 = nn.Linear(84, 10, bias=False)
73     # 定义前向传播，将tensor数据送入网络之后的计算图
74     def forward(self, x: torch.Tensor) -> torch.Tensor:
75         x = self.pool(F.relu(self.conv1(x)))
76         x = self.pool(F.relu(self.conv2(x)))
77         x = x.view(-1, 16 * 5 * 5)
78         x = F.relu(self.fc1(x))
79         x = F.relu(self.fc2(x))
80         x = self.fc3(x)
81         return x
82
83 net = Net().to(device)
84 # 到这里，我们定义好了网络和数据集的使用方式
85
86 # %%
87 from torch.utils.data import DataLoader
88 # 将训练集的dataloader送入网络进行训练，每次读取batchsize数量的数据
89 def train(model: nn.Module, dataloader: DataLoader):
90     criterion = nn.CrossEntropyLoss() # loss function (交叉熵损失函数)
91     optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) # optimizer，使用SGD优化
        器训练，第一个参数表示model的所有参数都会参与训练，学习率为0.001，动量为0.9
92
93     for epoch in range(2): # loop over the dataset multiple times。epoch指遍历所有训练集图像
        的次数。这里遍历所有训练图像两次
94
95         running_loss = 0.0
96         for i, data in enumerate(dataloader, 0): # dataloader是数据迭代器，从这里每次读出
            batchsize数量的数据。
97             # get the inputs; data is a list of [inputs, labels]
98             inputs, labels = data
99
100             inputs = inputs.to(device)
101             labels = labels.to(device)
102
103             # zero the parameter gradients
104             optimizer.zero_grad()
105
106             # forward + backward + optimize
107             outputs = model(inputs) # 将数据送进网络，得到结果
108             loss = criterion(outputs, labels) # 计算结果和标签的交叉熵损失函数

```

```

109         loss.backward() # 基于上述损失函数求解梯度
110         optimizer.step() # 更新参数
111
112         # print statistics
113         running_loss += loss.item() # 将loss的值取出来, 记录其变化
114         if i % 2000 == 1999: # print every 2000 mini-batches
115             print('%d, %5d] loss: %.3f' %
116                   (epoch + 1, i + 1, running_loss / 2000))
117             running_loss = 0.0
118
119         print('Finished Training')
120
121     def test(model: nn.Module, dataloader: DataLoader, max_samples=None) -> float:
122         correct = 0
123         total = 0
124         n_inferences = 0
125
126         with torch.no_grad(): # 测试时不用求解梯度, 因此可以设置不计算梯度的模式
127             for data in dataloader:
128                 images, labels = data
129
130                 images = images.to(device)
131                 labels = labels.to(device)
132
133                 outputs = model(images) # get 1 batch worth of image predictions (i.e. 4
134                 # predictions of 10 each)
135                 other, predicted = torch.max(outputs.data, 1) # other == values, predicted ==
136                 # indices
137                 total += labels.size(0)
138                 correct += (predicted == labels).sum().item()
139
140                 if max_samples:
141                     n_inferences += images.shape[0]
142                     if n_inferences > max_samples:
143                         break
144
145                 return 100 * correct / total
146
147     # %%
148     train(net, trainloader) # 进行训练, 将网络定义和数据迭代器传给train函数。
149
150     # %%
151     # Now that the CNN has been trained, let's test it on our test dataset.
152     score = test(net, testloader)
153     print('Accuracy of the network on the test images: {}'.format(score))
154
155     # %%
156     # Define a convenience function which we use to copy CNN's.
157     from copy import deepcopy
158
159     def copy_model(model: nn.Module) -> nn.Module:
160         result = deepcopy(model)
161
162         # Copy over the extra metadata we've collected which copy.deepcopy doesn't capture
163         if hasattr(model, 'input_activations'):
164             result.input_activations = deepcopy(model.input_activations)
165
166         for result_layer, original_layer in zip(result.children(), model.children()):
167             if isinstance(result_layer, nn.Conv2d) or isinstance(result_layer, nn.Linear):
168                 if hasattr(original_layer.weight, 'scale'):
169                     result_layer.weight.scale = deepcopy(original_layer.weight.scale)
170                 if hasattr(original_layer, 'activations'):

```

```

169         result_layer.activations = deepcopy(original_layer.activations)
170         if hasattr(original_layer, 'output_scale'):
171             result_layer.output_scale = deepcopy(original_layer.output_scale)
172
173     return result
174
175 # %%
176 # Question 1: Visualize Weights
177
178 # %%
179 ## Question 1.1:
180
181 import matplotlib.pyplot as plt
182 import numpy as np
183
184 name = ['conv1', 'conv2', 'fc1', 'fc2', 'fc3']
185 n = 0
186
187 for layer in net.children(): # 遍历类的成员
188     if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
189         weight = layer.weight.data.cpu().view(-1).numpy()
190         print(name[n] + ": the range is [{:.5f}, {:.5f}].format(weight.min(), weight.max())
191
192         # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
193         sigma3 = int(np.size(weight)*0.0027+0.5)
194         sigma3_value = np.abs(weight)[np.argpartition(np.abs(weight), -sigma3)[-sigma3]]
195         print(name[n] + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value,
196         weight.min()), min(sigma3_value, weight.max()))
197
198         plt.hist(weight, bins=50, facecolor="blue", edgecolor="black", alpha=0.7) # 画直方图
199         plt.xlabel("range")
200         plt.ylabel("frequency")
201         plt.title(name[n])
202         plt.show()
203         n += 1
204
205 # You can get a flattened vector of the weights of fc1 like this:
206 # fc1_weights = net.fc1.weight.data.cpu().view(-1)
207
208 # %%
209 ## Question 1.2: Explain which range you would prefer to use if you were to quantize each
210 layer's weights and wanted to strike a balance between the range of values that could be
211 expressed, and your precision.
212
213 # %%
214 # Question 2: Quantize Weights 首先我们把原来网络的参数copy到net_q2。
215
216 net_q2 = copy_model(net)
217
218 # %%
219 ## Question 2.1:
220
221 from typing import Tuple
222 # 确定scale进行量化
223 def quantized_weights(weights: torch.Tensor, bitwidth_W):
224     '''
225     Quantize the weights so that all values are integers between -128 and 127.
226     You may want to use the total range, 3-sigma range, or some other range when
227     deciding just what factors to scale the float32 values by.
228
229     Parameters:
230     weights (Tensor): The unquantized weights

```

```

228 #输入数据是尚未量化过的权重
229
230 Returns:
231 #返回数据是一个元组,(quantization result , scale )
232 (Tensor, float): A tuple with the following elements:
233     * The weights in quantized form, where every value is an integer
      between -128 and 127.
234     The "dtype" will still be "float", but the values themselves should
      all be integers.
235     * The scaling factor that your weights were multiplied by.
236     This value does not need to be an 8-bit integer.
237     '''
238
239 # modify the following code to try other range
240
241 W_border = 2**((bitwidth_W-1)
242 w = weights.data.cpu().view(-1).numpy() # 这里将tensor转入numpy只是为了之后的计算
243 sigma3 = int(np.size(w)*0.0027+0.5)
244 sigma3_value = np.abs(w)[np.argmaxpartition(np.abs(w), -sigma3)[-sigma3]]
245 weights = weights/sigma3_value*W_border
246 quan_weight = weights.round() # 转化为值最接近的整数
247
248 return torch.clamp(quan_weight, min=-W_border, max=W_border-1), W_border/sigma3_value #
      clamp是切顶函数,会把超过阈值的值自动替换成阈值
249
      # 我
      认为原来对scale的定义有问题,应该是反过来才对
250
251
252
253 # %%
254 def quantize_layer_weights(model: nn.Module, bitwidth_W):
255     for layer in model.children():
256         if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
257             q_layer_data, scale = quantized_weights(layer.weight.data, bitwidth_W)
258             q_layer_data = q_layer_data.to(device)
259
260             layer.weight.data = q_layer_data
261             layer.weight.scale = scale
262             print(scale) # 一共有2个卷积层和3个全连接层,因此会有5个scale
263
264             if (q_layer_data < -2**((bitwidth_W-1)).any() or (q_layer_data > (2**((bitwidth_W-1)
265 -1)).any()):
266                 raise Exception("Quantized weights of {} layer include values out of bounds
267 for an 8-bit signed integer".format(layer.__class__.__name__))
268                 if (q_layer_data != q_layer_data.round()).any():
269                     raise Exception("Quantized weights of {} layer include non-integer values".
270 format(layer.__class__.__name__))
271
272 quantize_layer_weights(net_q2, bitwidth_W)
273
274 # %%
275 ## Question 2.2:
276
277 #Record the accuracy change of the network after quantizing its weights. If you've done
278 everything correctly, the accuracy change should be negligible.
279
280 score = test(net_q2, testloader)
281 print('Accuracy of the network after quantizing all weights: {}'.format(score))
282
283 # %%
284 # Question 3: Visualize Activations

```

```

282
283 # Now that we have quantized the weights of the CNN, we must also quantize the activations (
      inputs and outputs to layers) traveling through it.
284 # But before doing so, let's analyze what values the activations take when travelling through
      the network.
285
286 # We provide convenience code which will record the values of every pixel of the outputs and
      inputs travelling through the neural network.
287 # (This is the initial CNN, where not even the weights had yet been quantized).
288 # We then profile these values when running on a subset of the training set.
289
290
291 # %%
292 # hook, 顾名思义, 其作用是把网络推理时的中间变量勾出来。通过hook函数, 我们可以统计中间变量的范
      围。
293 """
294 pytorch中的hook机制:
295 Pytorch会自动舍弃图计算中间结果,用hook以获取某些变量的中间结果.要及时删除,避免增加运行负载
296
297 Tensor级别:
298 register_hook(hook)->Tensor:为Tensor注册一个backward hook 获取变量梯度
299 hook遵循格式:hook(grad)->Tensor
300 nn.Module对象级别:
301 register_forward_hook(hook)
302 此时hook可以修改input和output却不影响forward结果,典型场景是提取中间层的输出特征,又不希望修改原
      有模型定义文件
303 格式为hook(layer,input,output)
304 """
305 def register_activation_profiling_hooks(model: Net):
306     model.input_activations = np.empty(0)
307     model.conv1.activations = np.empty(0)
308     model.conv2.activations = np.empty(0)
309     model.fc1.activations = np.empty(0)
310     model.fc2.activations = np.empty(0)
311     model.fc3.activations = np.empty(0)
312
313     model.profile_activations = True
314
315     def conv1_activations_hook(layer, x, y):
316         if model.profile_activations:
317             model.input_activations = np.append(model.input_activations, x[0].cpu().view(-1))
318     model.conv1.register_forward_hook(conv1_activations_hook)#寄存器暂时存下传递到下一层的
      activatio,下同
319
320     def conv2_activations_hook(layer, x, y):
321         if model.profile_activations:
322             model.conv1.activations = np.append(model.conv1.activations, x[0].cpu().view(-1))
323     model.conv2.register_forward_hook(conv2_activations_hook)
324
325     def fc1_activations_hook(layer, x, y):
326         if model.profile_activations:
327             model.conv2.activations = np.append(model.conv2.activations, x[0].cpu().view(-1))
328     model.fc1.register_forward_hook(fc1_activations_hook)
329
330
331     def fc2_activations_hook(layer, x, y):
332         if model.profile_activations:
333             model.fc1.activations = np.append(model.fc1.activations, x[0].cpu().view(-1))
334     model.fc2.register_forward_hook(fc2_activations_hook)
335
336     def fc3_activations_hook(layer, x, y):
337         if model.profile_activations:

```

```

338         model.fc2.activations = np.append(model.fc2.activations, x[0].cpu().view(-1))
339         model.fc3.activations = np.append(model.fc3.activations, y[0].cpu().view(-1))
340     model.fc3.register_forward_hook(fc3_activations_hook)
341
342
343     # %%
344     # Use the unquantized model to profile input and output activations on a subset of the
345     # training dataset.
346
347     net_q3 = copy_model(net)
348     register_activation_profiling_hooks(net_q3) #把各个中间层都记录下来
349
350     # Run through the training dataset again while profiling the input and output activations this
351     # time
352     # We don't actually have to perform gradient descent for this, so we can use the "test"
353     # function
354     test(net_q3, trainloader, max_samples=400) # 跑400次, 统计中间变量的数值范围
355     net_q3.profile_activations = False
356
357     # %%
358     ## Question 3.1:
359
360     # Plot histograms of the input images and the outputs of every convolutional and fully-
361     # connected layer.
362     # Record any observations you make about the distribution of the values.
363     # ADD YOUR CODE HERE
364
365     Activation_border = 2**((bitwidth_A-1))
366     input_activations = net_q3.input_activations # 输入数据
367     conv1_output_activations = net_q3.conv1.activations # 第一层卷积层的输出数据
368     conv2_output_activations = net_q3.conv2.activations # 第二层卷积层的输出数据
369     fc1_output_activations = net_q3.fc1.activations # 同上类推
370     fc2_output_activations = net_q3.fc2.activations
371     fc3_output_activations = net_q3.fc3.activations
372
373     # to plot distributions of activations
374     # Plot histograms of the following variables, and calculate their ranges and 3-sigma ranges:
375     # 拼接上面的数据成一个group, 方便循环遍历
376     activation_groups = [
377         (input_activations, "input_activations"),
378         (conv1_output_activations, "conv1_output_activations"),
379         (conv2_output_activations, "conv2_output_activations"),
380         (fc1_output_activations, "fc1_output_activations"),
381         (fc2_output_activations, "fc2_output_activations"),
382         (fc3_output_activations, "fc3_output_activations"),
383     ]
384
385     for activation_data, name in activation_groups: # 遍历类的成员
386         print(name + ": the range is [{:.5f}, {:.5f}]" .format(activation_data.min(),
387             activation_data.max()))
388
389         # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
390         sigma3 = int(np.size(activation_data)*0.0027+0.5)
391         sigma3_value = np.abs(activation_data)[np.argsort(np.abs(activation_data), -sigma3)[-
392             sigma3]]
393         print(name + ": the 3-sigma range is [{:.5f}, {:.5f}]" .format(max(-sigma3_value,
394             activation_data.min()), min(sigma3_value, activation_data.max())))
395
396     plt.hist(activation_data, bins=50, facecolor="green", edgecolor="black", alpha=0.7) # 画直
397     # 方图
398     plt.xlabel("range")
399     plt.ylabel("frequency")

```

```

392 plt.title(name+"_distribution")
393 plt.show()
394
395
396 # You can get a flattened vector of the weights of fc1 like this:
397 # fc1_weights = net.fc1.weight.data.cpu().view(-1)
398
399
400 # %%
401 ## Question 3.2:
402
403 # Additionally, record the range of the values, as well as their 3-sigma range (the difference
404 # between  $\mu + 3\sigma$  and  $\mu - 3\sigma$ ).
405 # For which layers is the 3-sigma range larger or smaller than the actual range?
406 # Then explain which range you would prefer to use if you were to quantize each layer's
407 # weights and wanted to strike a balance between the range of values that could be expressed
408 # , and your precision.
409 # Remember that you are plotting the activations *after* activation functions like ReLU have
410 # been applied, which means that you should not be worried if you find that your plots are
411 # asymmetric.
412
413 # %%
414 # Question 4: Quantize Activations
415
416 # Complete the `quantize_initial_input` and `quantize_activations` functions which calculate
417 # the scaling factors for the initial image which is input to the CNN, and the outputs of
418 # each layer, respectively.
419
420 # complete the `forward` function for the `NetQuantized` class.
421 # You will have to add code here to scale the outputs of each layer, and then to clamp the
422 # outputs of each layer to integers between -128 and 127 afterwards.
423
424 # %%
425 from typing import List
426
427 class NetQuantized(nn.Module):
428     def __init__(self, net_with_weights_quantized: nn.Module, bitwidth_A):
429         super(NetQuantized, self).__init__()
430
431         net_init = copy_model(net_with_weights_quantized) # 最初的网络copy 过来,用于初始化量化
432         网络
433
434         self.conv1 = net_init.conv1
435         self.pool = net_init.pool
436         self.conv2 = net_init.conv2
437         self.fc1 = net_init.fc1
438         self.fc2 = net_init.fc2
439         self.fc3 = net_init.fc3
440         self.activation_border = 2**(bitwidth_A-1)
441         print(self.activation_border)
442
443         for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
444             #逐层遍历,检查量化是否成功
445             def pre_hook(l, x):
446                 x = x[0]
447                 #异常处理:检查之前的量化是否将所有的数转化进规定的区间
448                 if (x < -self.activation_border).any() or (x > (self.activation_border-1)).any
449             ):
450                 raise Exception("Input to {} layer is out of bounds for an {}-bit signed
451 integer".format(l.__class__.__name__, bitwidth_A))
452                 if (x != x.round()).any():
453                     raise Exception("Input to {} layer has non-integer values".format(l.

```



```

__class__.__name__))
443
444     layer.register_forward_pre_hook(pre_hook)
445
446     # Calculate the scaling factor for the initial input to the CNN
447     self.input_activations = net_with_weights_quantized.input_activations
448     self.input_scale = NetQuantized.quantize_initial_input(self.input_activations, self.
activation_border)
449     # 以上2行计算了input activation 的scale和对应的quantization 版本
450     # Calculate the output scaling factors for all the layers of the CNN
451
452     n_w = []
453     activation_scale= [self.input_scale]
454
455     for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
456         # call quantize_activations and add your code here
457
458         #获取权重缩放因子
459         n_w.append(layer.weight.scale)
460
461         # 计算当前层激活的缩放因子
462         activation_scale.append( NetQuantized.quantize_initial_input(
463             layer.activations, 2*self.activation_border
464         ) )
465
466     #print(activation_scale)
467
468     n = 0
469     for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
470         layer.output_scale = NetQuantized.quantize_activations(n_w[n], activation_scale[n
], activation_scale[n + 1])
471         n = n + 1
472
473
474     # 注册前向检查钩子
475     for layer in self.conv1, self.conv2, self.fc1, self.fc2, self.fc3:
476         def pre_hook(l, x):
477             x = x[0]
478             if (x < -self.activation_border).any() or (x > (self.activation_border-1)).any
():
479                 raise Exception(f"Input to {l.__class__.__name__} layer is out of bounds")
480             if (x != x.round()).any():
481                 raise Exception(f"Input to {l.__class__.__name__} layer has non-integer
values")
482         layer.register_forward_pre_hook(pre_hook)
483
484     #计算输入的scale函数
485     @staticmethod
486     def quantize_initial_input(pixels: np.ndarray, activation_border:int):
487         '''
488         Calculate a scaling factor for the images that are input to the first layer of the CNN
489         .
490
491         Parameters:
492         pixels (ndarray): The values of all the pixels which were part of the input image
during training
493
494         Returns:
495         float: A scaling factor that the input should be multiplied by before being fed into
the first layer.
496         This value does not need to be an 8-bit integer.
497         '''

```

```

497     sigma3 = int(np.size(pixels)*0.0027+0.5)
498     sigma3_value = pixels[np.argmax(pixels,-sigma3)[-sigma3]]
499     max_value = np.max(np.abs(pixels))
500     scale = activation_border/max_value # you can modify the range here, 同样, 这个地方的
501     scale和论文定义的是和源代码倒数的关系
502     return scale
503
504 # 逐层激活送入量化
505 @staticmethod
506 def quantize_activations( n_w: float, pre_activation_scale: float, now_activation_scale:
507     float) -> float:
508     '''
509     Calculate a scaling factor to multiply the output of a layer by.
510
511     Parameters:
512     activations (ndarray): The values of all the pixels which have been output by this
513     layer during training
514     n_w (float): The scale by which the weights of this layer were multiplied as part of
515     the "quantize_weights" function you wrote earlier
516     n_initial_input (float): The scale by which the initial input to the neural network
517     was multiplied
518     ns [(float, float)]: A list of tuples, where each tuple represents the "weight scale
519     " and "output scale" (in that order) for every preceding layer
520     #用一个元组记录已经算好的所有网络的weight scale 和 output scale
521     Returns:
522     float: A scaling factor that the layer output should be multiplied by before being fed
523     into the first layer.
524     This value does not need to be an 8-bit integer.
525     '''
526     # ADD YOUR CODE HERE
527     scale = now_activation_scale / (pre_activation_scale * n_w)
528     return scale
529
530 def forward(self, x: torch.Tensor) -> torch.Tensor:
531     # You can access the output activation scales like this:
532     # fc1_output_scale = self.fc1.output_scale
533
534     # 这里其实是对定点数计算的模拟仿真, 因此每层的输出均需要手动乘以scaling factor, 并限制
535     在规定的表示范围内。
536     # output_scale:不同网络层之间传递整数重新压缩为8bit的scale(在第三周阅读论文中是定点浮
537     点数M)
538     # activation_scale:网络层内数据和真实值差距的scale
539     # weight_scale:权重量化值和真实值差距的scale
540     x = x*self.input_scale
541     x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border-1)
542     # 8bit是-128~127
543
544     x = self.pool(F.relu(self.conv1(x)))
545     x = x*self.conv1.output_scale
546     x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border-1)
547
548     x = self.pool(F.relu(self.conv2(x)))
549     x = x*self.conv2.output_scale
550     x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border-1)
551
552     x = x.view(-1, 16 * 5 * 5)
553     x = F.relu(self.fc1(x))
554     x = x*self.fc1.output_scale
555     x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border-1)
556

```

```

549
550     x = F.relu(self.fc2(x))
551     x = x*self.fc2.output_scale
552     x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border-1)
553
554
555     x = self.fc3(x)*self.fc3.output_scale
556     return x
557
558
559 # %%
560 #Merge the information from net_q2 and net_q3 together
561 #weight quantization & activation quantization
562 net_init = copy_model(net_q2)
563 net_init.input_activations = deepcopy(net_q3.input_activations)
564 for layer_init, layer_q3 in zip(net_init.children(), net_q3.children()):
565     if isinstance(layer_init, nn.Conv2d) or isinstance(layer_init, nn.Linear):
566         layer_init.activations = deepcopy(layer_q3.activations)
567
568 net_quantized = NetQuantized(net_init, bitwidth_A)
569
570
571 # %%
572 # Finally, record the accuracy of your network after both weights and activations have been
573 # quantized.
574 # If you've done everything right, you should still find almost no accuracy change.
575
576 score = test(net_quantized, testloader)
577 print('Accuracy of the network after quantizing both weights and activations: {}'.format(
578     score))
579
580 # %%
581 # Question 5: Quantize Biases 以下为选做内容
582
583 # Let us now update our CNN to include a bias in its final layer, *fc3*.
584 # We have already included code to create and train a new CNN called `net_with_bias`.
585
586 # Consider how a bias affects the equation for an unquantized layer:
587
588 #  $W * In + bias = Out$ 
589
590
591 # %%
592 ## Question 5.1:
593
594 # Suppose that we again quantized a biased layer with the same scaling factors we used in
595 # previous questions:  $n_W$ ,  $n_{In}$ , and  $n_{Out}$ .
596 # What would we scale bias by in this case?
597 # Write an equation in your lab report to describe the output of the quantized layer with a
598 # bias.
599
600 # Create a new network with a bias on *fc3*.
601
602 # %%
603 class NetWithBias(nn.Module):
604     def __init__(self):
605         super(NetWithBias, self).__init__()
606
607         self.conv1 = nn.Conv2d(3, 6, 5, bias=False)

```

```

607     self.pool = nn.MaxPool2d(2, 2)
608     self.conv2 = nn.Conv2d(6, 16, 5, bias=False)
609     self.fc1 = nn.Linear(16 * 5 * 5, 120, bias=False)
610     self.fc2 = nn.Linear(120, 84, bias=False)
611     self.fc3 = nn.Linear(84, 10, bias=True) # 最后一层的网络产生bias,这就是和之前的区别
612
613     def forward(self, x: torch.Tensor) -> torch.Tensor:
614         x = self.pool(F.relu(self.conv1(x)))
615         x = self.pool(F.relu(self.conv2(x)))
616         x = x.view(-1, 16 * 5 * 5)
617         x = F.relu(self.fc1(x))
618         x = F.relu(self.fc2(x))
619         x = self.fc3(x)
620         return x
621
622 net_with_bias = NetWithBias().to(device)
623
624 # %%
625 # Train and test the model
626
627 # %%
628 train(net_with_bias, trainloader)
629
630 # %%
631 score = test(net_with_bias, testloader)
632 print('Accuracy of the network (with a bias) on the test images: {}'.format(score))
633
634 # %%
635 register_activation_profiling_hooks(net_with_bias)
636 test(net_with_bias, trainloader, max_samples=400)
637 net_with_bias.profile_activations = False
638
639 # %%
640 net_with_bias_with_quantized_weights = copy_model(net_with_bias)
641 quantize_layer_weights(net_with_bias_with_quantized_weights, bitwidth_W)
642
643 score = test(net_with_bias_with_quantized_weights, testloader)
644 print('Accuracy of the network on the test images after all the weights are quantized but the
        bias isn\'t: {}'.format(score))
645
646 # %%
647 # Fill in the `quantized_bias` function in the `NetQuantizedWithBias` class.
648 # This function is meant to quantize the bias on the final layer of the CNN.
649 # Keep in mind that biases are typically quantized to 32-bits, so your bias values do not all
        have to be between -128 and 127 (though 32-bits is a bit conservative).
650
651 # %%
652 # slightly clearer bias bounds (32b signed integer)
653 #bias量化后的最大值、最小值
654 MIN_32B_SINT = -(2**31)
655 MAX_32B_SINT = (2**31) - 1
656
657 # %%
658 class NetQuantizedWithBias(NetQuantized):
659     def __init__(self, net_with_weights_quantized: nn.Module):
660         super(NetQuantizedWithBias, self).__init__(net_with_weights_quantized, bitwidth_A)
661
662         preceding_scales = [(layer.weight.scale, layer.output_scale) for layer in self.
            children() if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear)][:-1]
663         #print(preceding_scales)
664
665         self.fc3.bias.data = NetQuantizedWithBias.quantized_bias(

```

```

666         self.fc3.bias.data,
667         self.fc3.weight.scale,
668         self.input_scale,
669         preceding_scales
670     )
671
672
673     if (self.fc3.bias.data < MIN_32B_SINT).any() or (self.fc3.bias.data > MAX_32B_SINT).
any():
674         raise Exception("Bias has values which are out of bounds for an 32-bit signed
integer")
675         if (self.fc3.bias.data != self.fc3.bias.data.round()).any():
676             raise Exception("Bias has non-integer values")
677
678     @staticmethod
679     def quantized_bias(bias: torch.Tensor, n_w: float, n_initial_input: float, ns: List[Tuple[
float, float]]) -> torch.Tensor:
680         '''
681         Quantize the bias so that all values are integers between MIN_32B_SINT and
MAX_32B_SINT.
682
683         Parameters:
684         bias (Tensor): The floating point values of the bias
685         n_w (float): The scale by which the weights of this layer were multiplied
686         n_initial_input (float): The scale by which the initial input to the neural network
was multiplied
687         ns ([float, float]): A list of tuples, where each tuple represents the "weight scale
" and "output scale" (in that order) for every preceding layer
688
689         Returns:
690         Tensor: The bias in quantized form, where every value is an integer between
MIN_32B_SINT and MAX_32B_SINT.
691         The "dtype" will still be "float", but the values themselves should all be
integers.
692         '''
693
694         # ADD YOUR CODE HERE
695
696         #print(bias)
697         # 计算累积缩放因子: 初始输入缩放 * 所有前驱层的输出缩放
698
699         scale = n_initial_input
700         for weight_scale, output_scale in ns:
701             scale *= output_scale
702             scale *= weight_scale
703
704         # 总缩放因子 = 累积输入缩放 * 当前层权重缩放
705         scale = scale * n_w
706         print(scale)
707
708         # 量化偏置: bias * scale → 四舍五入 → 截断至32位整数范围
709         quan_bias = (bias * scale).round()
710         quan_bias = torch.clamp(quan_bias, min=MIN_32B_SINT, max=MAX_32B_SINT)
711         return quan_bias
712
713
714
715
716 # %%
717 net_quantized_with_bias = NetQuantizedWithBias(net_with_bias_with_quantized_weights)
718
719

```

```
720 # %%  
721 ## Question 5.3:  
722  
723 # What is your accuracy before and after quantizing CNN with the bias?  
724 # The accuracy change should ideally be negligible.  
725  
726 # %%  
727 score = test(net_quantized_with_bias, testloader)  
728 print('Accuracy of the network on the test images after all the weights and the bias are  
      quantized: {}'.format(score))
```

Listing 5: lab1_PTQ.ipynb

```

1  # %%
2  # 本次实验包括后量化的过程，主要在于学习如何对DNN的权重（weight）和激活层（activation）进行量
   化。
3
4  # %%
5  # 引用外部软件包
6  import torch # pytorch包
7  import torchvision # torchvision是pytorch官方提供的工具，包含很多常用数据集的封装等
8  import torchvision.transforms as transforms
9  import torch.nn as nn # nn 和下面的F均包含了主要的神经网络模块，包括卷积层、全连接层、池化层等
   等
10 import torch.nn.functional as F
11 import torch.optim as optim # 优化器
12
13 # %%
14 # 确保device为gpu，否则会花费过长时间。
15 if torch.cuda.is_available():
16     torch.cuda.set_device(0)
17     device = "cuda"
18 else:
19     device = torch.device('cpu')
20
21 # %%
22 # Load training and test data from the CIFAR10 dataset.
23 # torchvision是pytorch官方集成的工具库，包括学界常用的数据集和数据预处理方式。
24 transform = transforms.Compose(
25     [transforms.ToTensor(), # 将数据归一化，并转换成tensor的形式
26      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))] # 预处理数据的方式，normalize
   是将图像进行规则化。
27
28
29 trainset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数字系
   统设计/lab1/dataset/cifar10', train=True,
30                                         download=True, transform=transform) #定义了训练集，
   path是训练数据所在地址。
31 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
32                                           shuffle=True, num_workers=2) #定义数据迭代器，
   batchsize为一次迭代训练，送进去图像的数量（可以尝试调整，看看会发生什么）。shuffle表示要不
   要打乱训练图像的排序。
33
34 testset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数字系
   统设计/lab1/dataset/cifar10', train=False,
35                                         download=True, transform=transform)
36 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
37                                           shuffle=False, num_workers=2)
38
39 # %%
40 # Define a simple CNN that classifies CIFAR images.
41 # The network provided is similar to LeNet-5, and it has the following architecture:
42
43 # **Layer** |      **Type**      | **Input Shape** | **Output Shape** | **Activation**
44 #  conv1   | Convolutional   |    3x32x32     |    6x28x28      | ReLU
45 #  pool1   | Max pool       |    6x28x28     |    6x14x14      | None
46 #  conv2   | Convolutional   |    6x14x14     |   16x10x10      | ReLU
47 #  pool2   | Max pool       |   16x10x10     |   16x5x5        | None
48 #   fc1    | Fully-connected |        400      |        120       | ReLU
49 #   fc2    | Fully-connected |        120      |         84        | ReLU
50 #   fc3    | Fully-connected |         84      |          10       | None
51
52 # None of the layers in the network have a bias associated with them.
53 # This makes them easier to quantize.

```

```

54 # Towards the end of this assignment, we will add biases to the final layer and quantize it as
    well.
55
56 # %%
57 # 重新定义conv函数与fc函数
58 import math
59 def TorchRound():
60     """
61     Apply STE to clamp function.
62     """
63     class identity_quant(torch.autograd.Function):
64         @staticmethod
65         def forward(ctx, input):
66             out = torch.round(input)
67             return out
68
69         @staticmethod
70         def backward(ctx, grad_output):
71             return grad_output
72
73     return identity_quant().apply
74
75 def quant_max(tensor):
76     """
77     Returns the max value for symmetric quantization.
78     """
79     return torch.abs(tensor.detach()).max() + 1e-8 # 防止除0的情况出现
80
81 class quant_weight(nn.Module):
82     """
83     Quantization function for quantize weight with maximum.
84     """
85
86     def __init__(self, k_bits=8):
87         super(quant_weight, self).__init__()
88         self.k_bits = k_bits
89         self.qmax = 2. ** (k_bits - 1) - 1.
90         self.round = TorchRound()
91
92     def forward(self, input):
93         max_val = quant_max(input)
94         weight = input * self.qmax / max_val
95         q_weight = self.round(weight)
96         q_weight = q_weight * max_val / self.qmax # 通过这种方式引入量化误差, 从而实现QAT
97         return q_weight
98
99 class QuantConv2d(nn.Module):
100     """
101     A convolution layer with quantized weight.
102     """
103
104     def __init__(self, in_channels, out_channels, kernel_size, stride=1,
105                  padding=0, dilation=1, groups=1, bias=False, k_bits=8): # 默认使用bias=
106         # False的情况
107         super(QuantConv2d, self).__init__()
108         self.weight = nn.Parameter(torch.Tensor(out_channels, in_channels, kernel_size,
109         kernel_size))
110         self.stride = stride
111         self.padding = padding
112         self.dilation = dilation
113         self.groups = groups
114         self.in_channels = in_channels

```



```

113     self.kernel_size = (kernel_size, kernel_size)
114     self.bias_flag = bias
115     if self.bias_flag:
116         self.bias = nn.Parameter(torch.Tensor(out_channels))
117         nn.init.constant_(self.bias)
118     else:
119         self.register_parameter('bias', None)
120     self.k_bits = k_bits
121     self.quant_weight = quant_weight(k_bits = k_bits)
122     self.output = None
123     nn.init.kaiming_normal_(self.weight) # 初始化参数
124
125
126     def forward(self, input, order=None):
127         return F.conv2d(input, self.quant_weight(self.weight), self.bias, self.stride, self.
padding, self.dilation, self.groups)
128
129 class QuantFC(nn.Module):
130     """
131     A Fully Connected layer with quantized weight.
132     """
133
134     def __init__(self, in_features, out_features, bias=False, k_bits=8): # 默认使用bias=False
的情况
135         super(QuantFC, self).__init__()
136         self.weight = nn.Parameter(torch.Tensor(out_features, in_features))
137         self.bias_flag = bias
138         if self.bias_flag:
139             self.bias = nn.Parameter(torch.Tensor(out_features))
140         else:
141             self.register_parameter('bias', None)
142         self.k_bits = k_bits
143         self.quant_weight = quant_weight(k_bits = k_bits)
144         nn.init.kaiming_normal_(self.weight) # 初始化参数, 何恺明发明
145
146
147     def forward(self, input, order=None):
148         return F.linear(input, self.quant_weight(self.weight), self.bias)
149
150 # %%
151 class quant_act(nn.Module):
152     """
153     Quantization function for quantize activation with parameterized max scale.
154     """
155     def __init__(self, k_bits=8, decay=0.995):
156         super(quant_act, self).__init__()
157         self.k_bits = k_bits
158         self.qmax = 2. ** (self.k_bits - 1) - 1.
159         self.round = TorchRound()
160         self.decay = decay #平滑因子, 用于EMA, 能够更敏感地反映最新的数据变化
161         self.alpha = nn.Parameter(torch.Tensor(10)) #将激活值限制在 [-alpha, alpha] 范围内,
避免异常值破坏量化。
162         self.register_buffer('max_val', torch.ones(1))
163
164     def _ema(self, x): #通过指数移动平均 (EMA) 跟踪激活值的最大绝对值范围
165         act = x.reshape(x.shape[0], -1)
166         # print(act.shape)
167         max_val = torch.mean(torch.max(act, dim=1)[0])
168
169         self.max_val = (1.0-self.decay) * max_val + self.decay * self.max_val
170
171     def forward(self, x):

```

```

172         act = x
173         self._ema(x)
174         self.alpha.data = self.max_val.unsqueeze(0)
175         act = torch.max(torch.min(x, self.alpha), -self.alpha)
176
177         act = act * self.qmax / self.alpha
178         q_act = self.round(act)
179         q_act = q_act * self.alpha / self.qmax # 引入量化误差
180
181         return q_act
182
183 # %%
184 # 网络定义, conv2d是卷积层, linear是全连接层。forward下定义了计算图, 显示送进来的图像(x)会怎么处理。
185 class Net(nn.Module):
186     # 网络结构:2层卷积层、1层最大值池化层、3层全连接层, 留意这些参数的内涵
187     def __init__(self):
188         super(Net, self).__init__()
189         self.conv1 = QuantConv2d(3, 6, 5, bias=False, k_bits=8) # Weight for 8bit
190         self.pool = nn.MaxPool2d(2, 2)
191         self.conv2 = QuantConv2d(6, 16, 5, bias=False, k_bits=8)
192         self.fc1 = QuantFC(16 * 5 * 5, 120, bias=False, k_bits=8)
193         self.fc2 = QuantFC(120, 84, bias=False, k_bits=8)
194         self.fc3 = QuantFC(84, 10, bias=False, k_bits=8)
195
196         self.quant_act1 = quant_act(k_bits=8) # activation for 8 bit
197         self.quant_act2 = quant_act(k_bits=8)
198         self.quant_act3 = quant_act(k_bits=8)
199         self.quant_act4 = quant_act(k_bits=8)
200         # self.quant_act5 = quant_act(k_bits=8)
201     # 定义前向传播, 将tensor数据送入网络之后的计算图
202     def forward(self, x: torch.Tensor) -> torch.Tensor:
203         x = self.pool(F.relu(self.conv1(x)))
204         x = self.quant_act1(x)
205
206         x = self.pool(F.relu(self.conv2(x)))
207         x = self.quant_act2(x)
208
209         x = x.view(-1, 16 * 5 * 5)
210         x = F.relu(self.fc1(x))
211         x = self.quant_act3(x)
212
213         x = F.relu(self.fc2(x))
214         x = self.quant_act4(x)
215
216         x = self.fc3(x)
217         # x = self.quant_act5(x)
218         return x
219
220 net = Net().to(device)
221 # 到这里, 我们定义好了网络和数据集的使用方式
222
223 # %%
224 from torch.utils.data import DataLoader
225 # 将训练集的dataloader送入网络进行训练, 每次读取batchsize数量的数据
226 def train(model: nn.Module, dataloader: DataLoader):
227     criterion = nn.CrossEntropyLoss() # loss function (交叉熵损失函数)
228     optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) #optimizer, 使用SGD优化器训练, 第一个参数表示model的所有参数都会参与训练, 学习率为0.001, 动量为0.9
229
230     for epoch in range(2): # loop over the dataset multiple times. epoch指遍历所有训练集图像的
        # 次数。这里遍历所有训练图像三次

```

```

231
232     running_loss = 0.0
233     for i, data in enumerate(dataloader, 0): # dataloader是数据迭代器，从这里每次读出
        # 将数据送入网络，得到结果
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
236
237         inputs = inputs.to(device)
238         labels = labels.to(device)
239
240         # zero the parameter gradients
241         optimizer.zero_grad()
242
243         # forward + backward + optimize
244         outputs = model(inputs) # 将数据送入网络，得到结果
245         loss = criterion(outputs, labels) # 计算结果和标签的交叉熵损失函数
246         loss.backward() # 基于上述损失函数求解梯度
247         optimizer.step() # 更新参数
248
249         # print statistics
250         running_loss += loss.item() # 将loss的值取出来，记录其变化
251         if i % 2000 == 1999: # print every 2000 mini-batches
252             print('[%d, %5d] loss: %.3f' %
253                   (epoch + 1, i + 1, running_loss / 2000))
254             running_loss = 0.0
255
256     print('Finished Training')
257
258 def test(model: nn.Module, dataloader: DataLoader, max_samples=None) -> float:
259     correct = 0
260     total = 0
261     n_inferences = 0
262
263     with torch.no_grad(): # 测试时不用求解梯度，因此可以设置不计算梯度的模式
264         for data in dataloader:
265             images, labels = data
266
267             images = images.to(device)
268             labels = labels.to(device)
269
270             outputs = model(images) # get 1 batch worth of image predictions (i.e. 4
        # predictions of 10 each)
271             other, predicted = torch.max(outputs.data, 1) # other == values, predicted ==
        # indices
272             total += labels.size(0)
273             correct += (predicted == labels).sum().item()
274
275             if max_samples:
276                 n_inferences += images.shape[0]
277                 if n_inferences > max_samples:
278                     break
279
280     return 100 * correct / total
281
282 # %%
283 train(net, trainloader) # 进行训练，将网络定义和数据迭代器传给train函数。
284
285 # %%
286 # Now that the CNN has been trained, let's test it on our test dataset.
287 score = test(net, testloader)
288 print('Accuracy of the network on the test images: {}'.format(score))
289

```

```

290 # %%
291 # Define a convenience function which we use to copy CNN's.
292 from copy import deepcopy
293
294 def copy_model(model: nn.Module) -> nn.Module:
295     result = deepcopy(model)
296
297     # Copy over the extra metadata we've collected which copy.deepcopy doesn't capture
298     if hasattr(model, 'input_activations'):
299         result.input_activations = deepcopy(model.input_activations)
300
301     for result_layer, original_layer in zip(result.children(), model.children()):
302         if isinstance(result_layer, nn.Conv2d) or isinstance(result_layer, nn.Linear):
303             if hasattr(original_layer.weight, 'scale'):
304                 result_layer.weight.scale = deepcopy(original_layer.weight.scale)
305             if hasattr(original_layer, 'activations'):
306                 result_layer.activations = deepcopy(original_layer.activations)
307             if hasattr(original_layer, 'output_scale'):
308                 result_layer.output_scale = deepcopy(original_layer.output_scale)
309
310     return result
311
312 # %%
313 # Visualize Weights
314
315 # %%
316 import matplotlib.pyplot as plt
317 import numpy as np
318
319 name = ['conv1', 'conv2', 'fc1', 'fc2', 'fc3']
320 n = 0
321
322 for layer in net.children(): # 遍历类的成员
323     if isinstance(layer, QuantConv2d) or isinstance(layer, QuantFC):
324         weight = layer.weight.data.cpu().view(-1).numpy()
325         print(name[n] + ": the range is [{:.5f}, {:.5f}].format(weight.min(), weight.max()))
326
327         # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
328         sigma3 = int(np.size(weight)*0.0027)
329         sigma3_value = np.abs(weight)[np.argsort(np.abs(weight), -sigma3)[-sigma3]]
330         print(name[n] + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value,
331         weight.min()), min(sigma3_value, weight.max()))
332
333         plt.hist(weight, bins=50, facecolor="blue", edgecolor="black", alpha=0.7) # 画直方图
334         plt.xlabel("range")
335         plt.ylabel("frequency")
336         plt.title(name[n])
337         plt.show()
338         n += 1
339
340 # You can get a flattened vector of the weights of fc1 like this:
341 # fc1_weights = net.fc1.weight.data.cpu().view(-1)

```

Listing 6: lab1_QAT.ipynb

```

1 # %%
2 # 本次实验包括后量化的过程，主要在于学习如何对DNN的权重（weight）和激活层（activation）进
  行量化。
3
4 # %%
5 # 引用外部软件包
6 import torch # pytorch包

```

```

7 import torchvision # torchvision是pytorch官方提供的工具，包含很多常用数据集的封装等
8 import torchvision.transforms as transforms
9 import torch.nn as nn # nn 和下面的F均包含了主要的神经网络模块，包括卷积层、全连接层、池化
  层等等
10 import torch.nn.functional as F
11 import torch.optim as optim # 优化器
12
13 # %%
14 # 确保device为gpu，否则会花费过长时间。
15 if torch.cuda.is_available():
16     torch.cuda.set_device(0) # 0为选择显卡id
17     device = "cuda"
18     print('using cuda')
19 else:
20     device = torch.device('cpu')
21     print('using cpu')
22
23 # 设置量化位宽
24 bitwidth_A = 8
25 bitwidth_W = 8
26
27 # %%
28 # Load training and test data from the CIFAR10 dataset.
29 # torchvision是pytorch官方集成的工具库，包括学界常用的数据集和数据预处理方式。
30 transform = transforms.Compose(
31     [transforms.ToTensor(), # 将数据归一化，并转换成tensor的形式
32      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))] # 预处理数据的方式，
33     normalize是将图像进行规则化。
34
35 trainset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset', train=True,
36                                         download=True, transform=transform) #定义了训练
37     集，path是训练数据所在地址，指定位置自动下载。
38
39 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
40                                           shuffle=True, num_workers=2) #定义数据迭代器，
41     batchsize为一次迭代训练，送进去图像的数量（可以尝试调整，看看会发生什么）。shuffle表示要不
42     要打乱训练图像的排序。
43
44 testset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset', train=False,
45                                         download=True, transform=transform)
46
47 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
48                                           shuffle=False, num_workers=2)
49
50 # %%
51 # Define a simple CNN that classifies CIFAR images.
52 # The net has the following architecture:
53
54 # **Layer** | **Type** | **Input Shape** | **Output Shape** | **Activation**
55 # conv1 | Convolutional | 3x32x32 | 16x32x32 | ReLU
56 # conv2 | Convolutional | 16x32x32 | 16x32x32 | ReLU
57 # pool1 | Max pool | 16x32x32 | 16x16x16 | None
58 # conv3 | Convolutional | 16x16x16 | 32x16x16 | ReLU
59 # conv4 | Convolutional | 32x16x16 | 32x16x16 | ReLU
60 # pool2 | Max pool | 32x16x16 | 32x8x8 | None
61 # fc1 | Fully-connected | 32x8x8 | 128 | ReLU
62 # fc2 | Fully-connected | 128 | 10 | ReLU
63
64 # None of the layers in the network have a bias associated with them.
65 # This makes them easier to quantize.
66 # Towards the end of this assignment, we will add biases to the final layer and quantize
  it as well.

```

```

61
62 # %%
63 # 网络定义，conv2d是卷积层，linear是全连接层。forward下定义了计算图，显示送进来的图像(x)
    会怎么处理。
64 class Net(nn.Module):
65     #网络结构:4层卷积层、2层最大值池化层、4层全连接层，留意这些参数的内涵
66     def __init__(self):
67         super(Net, self).__init__()
68         self.conv1 = nn.Conv2d(3, 16, 3, padding='same', bias=False)
69         self.conv2 = nn.Conv2d(16, 16, 3, padding='same', bias=False)
70         self.pool = nn.MaxPool2d(2, 2) # run after each conv (hence the 3x3 FC layer)
71         self.conv3 = nn.Conv2d(16, 32, 3, padding='same', bias=False)
72         self.conv4 = nn.Conv2d(32, 32, 3, padding='same', bias=False)
73         self.pool2 = nn.AvgPool2d(2,2)
74         self.fc1 = nn.Linear(32 * 8 * 8, 128, bias=False)
75         self.fc2 = nn.Linear(128, 10, bias=False)
76     #定义前向传播,将tensor数据送入网络之后的计算图
77     def forward(self, x: torch.Tensor) -> torch.Tensor:
78         x = F.relu(self.conv1(x))
79         x = F.relu(self.conv2(x))
80         x = self.pool(x)
81         x = F.relu(self.conv3(x))
82         x = F.relu(self.conv4(x))
83         x = self.pool(x)
84         x = x.view(-1, 32 * 8 * 8)
85         x = F.relu(self.fc1(x))
86         x = self.fc2(x)
87         return x
88
89 net = Net().to(device)
90 # 到这里，我们定义好了网络和数据集的使用方式
91
92 # %%
93 from torch.utils.data import DataLoader
94 #将训练集的dataloader送入网络进行训练,每次读取batchsize数量的数据
95 def train(model: nn.Module, dataloader: DataLoader):
96     criterion = nn.CrossEntropyLoss() # loss function (交叉熵损失函数)
97     optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) #optimizer, 使用SGD
    优化器训练，第一个参数表示model的所有参数都会参与训练，学习率为0.001，动量为0.9
98
99     for epoch in range(10): # loop over the dataset multiple times。epoch指遍历所有训练集
    图像的个数。这里遍历所有训练图像两次
100
101         running_loss = 0.0
102         for i, data in enumerate(dataloader, 0): # dataloader是数据迭代器，从这里每次读出
    batchsize数量的数据。
103             # get the inputs; data is a list of [inputs, labels]
104             inputs, labels = data
105
106             inputs = inputs.to(device)
107             labels = labels.to(device)
108
109             # zero the parameter gradients
110             optimizer.zero_grad()
111
112             # forward + backward + optimize
113             outputs = model(inputs) # 将数据送进网络，得到结果
114             loss = criterion(outputs, labels) # 计算结果和标签的交叉熵损失函数
115             loss.backward() # 基于上述损失函数求解梯度
116             optimizer.step() # 更新参数
117
118             # print statistics

```

```

119         running_loss += loss.item() # 将loss的值取出来, 记录其变化
120         if i % 2000 == 1999:         # print every 2000 mini-batches
121             print('%d, %5d] loss: %.3f' %
122                   (epoch + 1, i + 1, running_loss / 2000))
123             running_loss = 0.0
124
125     print('Finished Training')
126
127     def test(model: nn.Module, dataloader: DataLoader, max_samples=None) -> float:
128         correct = 0
129         total = 0
130         n_inferences = 0
131
132         with torch.no_grad(): # 测试时不用求解梯度, 因此可以设置不计算梯度的模式
133             for data in dataloader:
134                 images, labels = data
135
136                 images = images.to(device)
137                 labels = labels.to(device)
138
139                 outputs = model(images) # get 1 batch worth of image predictions (i.e. 4
140                 predictions of 10 each)
141                 other, predicted = torch.max(outputs.data, 1) # other == values, predicted ==
142                 indicies
143                 total += labels.size(0)
144                 correct += (predicted == labels).sum().item()
145
146                 if max_samples:
147                     n_inferences += images.shape[0]
148                     if n_inferences > max_samples:
149                         break
150
151             return 100 * correct / total
152
153     # %%
154     train(net, trainloader) # 进行训练, 将网络定义和数据迭代器传给train函数。
155
156     # %%
157     # Now that the CNN has been trained, let's test it on our test dataset.
158     score = test(net, testloader)
159     print('Accuracy of the network on the test images: {}'.format(score))
160
161     # %%
162     # Define a convenience function which we use to copy CNN's.
163     from copy import deepcopy
164
165     def copy_model(model: nn.Module) -> nn.Module:
166         result = deepcopy(model)
167
168         # Copy over the extra metadata we've collected which copy.deepcopy doesn't capture
169         if hasattr(model, 'input_activations'):
170             result.input_activations = deepcopy(model.input_activations)
171
172         for result_layer, original_layer in zip(result.children(), model.children()):
173             if isinstance(result_layer, nn.Conv2d) or isinstance(result_layer, nn.Linear):
174                 if hasattr(original_layer, 'scale'):
175                     result_layer.weight.scale = deepcopy(original_layer.weight.scale)
176                 if hasattr(original_layer, 'activations'):
177                     result_layer.activations = deepcopy(original_layer.activations)
178                 if hasattr(original_layer, 'output_scale'):
179                     result_layer.output_scale = deepcopy(original_layer.output_scale)

```

```

179         return result
180
181     # %%
182     # Question 1: Visualize Weights
183
184     # %%
185     ## Question 1.1:
186
187     import matplotlib.pyplot as plt
188     import numpy as np
189
190     name = ['conv1', 'conv2', 'conv3', 'conv4', 'fc1', 'fc2']
191     n = 0
192
193     for layer in net.children(): # 遍历类的成员
194         if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
195             weight = layer.weight.data.cpu().view(-1).numpy()
196             print(name[n] + ": the range is [{:.5f}, {:.5f}].format(weight.min(), weight.max
197             ()))
198
199             # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
200             sigma3 = int(np.size(weight)*0.0027+0.5)
201             sigma3_value = np.abs(weight)[np.argpartition(np.abs(weight), -sigma3)[-sigma3]]
202             print(name[n] + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value
203             , weight.min()), min(sigma3_value, weight.max()))
204
205             plt.hist(weight, bins=50, facecolor="blue", edgecolor="black", alpha=0.7) # 画直方
206             图
207             plt.xlabel("range")
208             plt.ylabel("frequency")
209             plt.title(name[n])
210             plt.show()
211             n += 1
212
213     # You can get a flattened vector of the weights of fc1 like this:
214     # fc1_weights = net.fc1.weight.data.cpu().view(-1)
215
216     # %%
217     ## Question 1.2: Explain which range you would prefer to use if you were to quantize each
218     layer's weights and wanted to strike a balance between the range of values that could be
219     expressed, and your precision.
220
221     # %%
222     # Question 2: Quantize Weights 首先我们把原来网络的参数copy到net_q2。
223
224     net_q2 = copy_model(net)
225
226     # %%
227     ## Question 2.1:
228
229     from typing import Tuple
230     # 确定scale进行量化
231     def quantized_weights(weights: torch.Tensor, bitwidth_W):
232         """
233         Quantize the weights so that all values are integers between -128 and 127.
234         You may want to use the total range, 3-sigma range, or some other range when
235         deciding just what factors to scale the float32 values by.
236
237         Parameters:
238         weights (Tensor): The unquantized weights
239         #输入数据是尚未量化过的权重

```



```

236     Returns:
237     # 返回数据是一个元组,(quantization result , scale )
238     (Tensor, float): A tuple with the following elements:
239         * The weights in quantized form, where every value is an integer
240         between -128 and 127.
241         The "dtype" will still be "float", but the values themselves
242         should all be integers.
243         * The scaling factor that your weights were multiplied by.
244         This value does not need to be an 8-bit integer.
245     '''
246
247     # modify the following code to try other range
248
249     W_border = 2**((bitwidth_W-1))
250     w = weights.data.cpu().view(-1).numpy() # 这里将tensor转入numpy只是为了之后的计算
251     sigma3 = int(np.size(w)*0.0027+0.5)
252     sigma3_value = np.abs(w)[np.argmax(np.abs(w), -sigma3)[-sigma3]]
253     weights = weights/sigma3_value*W_border
254     quan_weight = weights.round() # 转化为值最接近的整数
255
256     return torch.clamp(quan_weight, min=-W_border, max=W_border-1), W_border/sigma3_value
257 # clamp是切顶函数,会把超过阈值的值自动替换成阈值
258
259 # 我认为原来对scale的定义有问题,应该是反过来才对
260
261 # %%
262 def quantize_layer_weights(model: nn.Module, bitwidth_W):
263     for layer in model.children():
264         if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
265             q_layer_data, scale = quantized_weights(layer.weight.data, bitwidth_W)
266             q_layer_data = q_layer_data.to(device)
267
268             layer.weight.data = q_layer_data
269             layer.weight.scale = scale
270             print(scale) # 一共有4个卷积层和2个全连接层,因此会有6个scale
271
272             if (q_layer_data < -2**((bitwidth_W-1))).any() or (q_layer_data > (2**((
273 bitwidth_W-1)-1))).any():
274                 raise Exception("Quantized weights of {} layer include values out of
275 bounds for an 8-bit signed integer".format(layer.__class__.__name__))
276             if (q_layer_data != q_layer_data.round()).any():
277                 raise Exception("Quantized weights of {} layer include non-integer values"
278 .format(layer.__class__.__name__))
279
280 quantize_layer_weights(net_q2, bitwidth_W)
281
282 # %%
283 ## Question 2.2:
284
285 #Record the accuracy change of the network after quantizing its weights. If you've done
286 everything correctly, the accuracy change should be negligible.
287
288 score = test(net_q2, testloader)
289 print('Accuracy of the network after quantizing all weights: {}'.format(score))
290
291 # %%
292 # Question 3: Visualize Activations
293
294 # Now that we have quantized the weights of the CNN, we must also quantize the activations

```

```

    (inputs and outputs to layers) traveling through it.
290 # But before doing so, let's analyze what values the activations take when travelling
    through the network.
291
292 # We provide convenience code which will record the values of every pixel of the outputs
    and inputs travelling through the neural network.
293 # (This is the initial CNN, where not even the weights had yet been quantized).
294 # We then profile these values when running on a subset of the training set.
295
296
297 # %%
298 # hook, 顾名思义, 其作用是把网络推理时的中间变量勾出来。通过hook函数, 我们可以统计中间变量
    的范围。
299 """
300 pytorch中的hook机制:
301 Pytorch会自动舍弃图计算中间结果,用hook以获取某些变量的中间结果.要及时删除,避免增加运行负载
302
303 Tensor级别:
304 register_hook(hook)->Tensor:为Tensor注册一个backward hook 获取变量梯度
305 hook遵循格式:hook(grad)->Tensor
306 nn.Module对象级别:
307 register_forward_hook(hook)
308 此时hook可以修改input和output却不影响forward结果,典型场景是提取中间层的输出特征,又不希望修
    改原有模型定义文件
309 格式为hook(layer,input,output)
310 """
311 def register_activation_profiling_hooks(model: Net):
312     model.input_activations = np.empty(0)
313     model.conv1.activations = np.empty(0)
314     model.conv2.activations = np.empty(0)
315     model.conv3.activations = np.empty(0)
316     model.conv4.activations = np.empty(0)
317     model.fc1.activations = np.empty(0)
318     model.fc2.activations = np.empty(0)
319     #model.fc3.activations = np.empty(0)
320
321     model.profile_activations = True
322
323     def conv1_activations_hook(layer, x, y):
324         if model.profile_activations:
325             model.input_activations = np.append(model.input_activations, x[0].cpu().view
    (-1))
326             model.conv1.register_forward_hook(conv1_activations_hook)#寄存器暂时存下传递到下一层的
    activatio,下同
327
328     def conv2_activations_hook(layer, x, y):
329         if model.profile_activations:
330             model.conv1.activations = np.append(model.conv1.activations, x[0].cpu().view
    (-1))
331             model.conv2.register_forward_hook(conv2_activations_hook)
332
333     def conv3_activations_hook(layer, x, y):
334         if model.profile_activations:
335             model.conv2.activations = np.append(model.conv2.activations, x[0].cpu().view
    (-1))
336             model.conv3.register_forward_hook(conv3_activations_hook)
337
338     def conv4_activations_hook(layer, x, y):
339         if model.profile_activations:
340             model.conv3.activations = np.append(model.conv3.activations, x[0].cpu().view
    (-1))
341             model.conv4.register_forward_hook(conv4_activations_hook)

```

```

342
343     def fc1_activations_hook(layer, x, y):
344         if model.profile_activations:
345             model.conv4.activations = np.append(model.conv4.activations, x[0].cpu().view
346 (-1))
347     model.fc1.register_forward_hook(fc1_activations_hook)
348
349     def fc2_activations_hook(layer, x, y):
350         if model.profile_activations:
351             model.fc1.activations = np.append(model.fc1.activations, x[0].cpu().view(-1))
352             model.fc2.activations = np.append(model.fc2.activations, y[0].cpu().view(-1))
353     model.fc2.register_forward_hook(fc2_activations_hook)
354
355     #def fc3_activations_hook(layer, x, y):
356     #     if model.profile_activations:
357     #         model.fc2.activations = np.append(model.fc2.activations, x[0].cpu().view(-1))
358     #         model.fc3.activations = np.append(model.fc3.activations, y[0].cpu().view(-1))
359     #model.fc3.register_forward_hook(fc3_activations_hook)
360
361
362     # %%
363     # Use the unquantized model to profile input and output activations on a subset of the
364     training dataset.
365
366     net_q3 = copy_model(net)
367     register_activation_profiling_hooks(net_q3) #把各个中间层都记录下来
368
369     # Run through the training dataset again while profiling the input and output activations
370     this time
371     # We don't actually have to perform gradient descent for this, so we can use the "test"
372     function
373     test(net_q3, trainloader, max_samples=400) # 跑400次, 统计中间变量的数值范围
374     net_q3.profile_activations = False
375
376     # %%
377     ## Question 3.1:
378
379     # Plot histograms of the input images and the outputs of every convolutional and fully-
380     connected layer.
381     # Record any observations you make about the distribution of the values.
382     # ADD YOUR CODE HERE
383     Activation_border = 2*(bitwidth_A-1)
384     input_activations = net_q3.input_activations # 输入数据
385     conv1_output_activations = net_q3.conv1.activations # 第一层卷积层的输出数据
386     conv2_output_activations = net_q3.conv2.activations # 第二层卷积层的输出数据
387     conv3_output_activations = net_q3.conv3.activations # 第三层卷积层的输出数据
388     conv4_output_activations = net_q3.conv4.activations # 第四层卷积层的输出数据
389     fc1_output_activations = net_q3.fc1.activations # 同上类推
390     fc2_output_activations = net_q3.fc2.activations
391     #fc3_output_activations = net_q3.fc3.activations
392
393     #print(conv1_output_activations)
394
395     # to plot distributions of activations
396     # Plot histograms of the following variables, and calculate their ranges and 3-sigma
397     ranges:
398     # 拼接上面的数据成一个group, 方便循环遍历
399     activation_groups = [
400         (input_activations, "input_activations"),
401         (conv1_output_activations, "conv1_output_activations"),
402         (conv2_output_activations, "conv2_output_activations"),

```

```

398     (conv3_output_activations, "conv3_output_activations"),
399     (conv4_output_activations, "conv4_output_activations"),
400     (fc1_output_activations, "fc1_output_activations"),
401     (fc2_output_activations, "fc2_output_activations"),
402 ]
403
404 for activation_data, name in activation_groups: # 遍历类的成员
405     print(name + ": the range is [{:.5f}, {:.5f}].format(activation_data.min(),
406           activation_data.max()))
407
408     # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
409     sigma3 = int(np.size(activation_data)*0.0027+0.5)
410     sigma3_value = np.abs(activation_data)[np.argpartition(np.abs(activation_data), -
411           sigma3)[-sigma3]]
412     print(name + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value,
413           activation_data.min()), min(sigma3_value, activation_data.max()))
414
415     plt.hist(activation_data, bins=50, facecolor="green", edgecolor="black", alpha=0.7) #
416     画直方图
417     plt.xlabel("range")
418     plt.ylabel("frequency")
419     plt.title(name+"_distribution")
420     plt.show()
421
422 # You can get a flattened vector of the weights of fc1 like this:
423 #   fc1_weights = net.fc1.weight.data.cpu().view(-1)
424
425 # %%
426 ## Question 3.2:
427
428 # Additionally, record the range of the values, as well as their 3-sigma range (the
429 # difference between  $\mu + 3\sigma$  and  $\mu - 3\sigma$ ).
430 # For which layers is the 3-sigma range larger or smaller than the actual range?
431 # Then explain which range you would prefer to use if you were to quantize each layer's
432 # weights and wanted to strike a balance between the range of values that could be expressed
433 # , and your precision.
434 # Remember that you are plotting the activations *after* activation functions like ReLU
435 # have been applied, which means that you should not be worried if you find that your plots
436 # are asymmetric.
437
438 # %%
439 # Question 4: Quantize Activations
440
441 # Complete the `quantize_initial_input` and `quantize_activations` functions which
442 # calculate the scaling factors for the initial image which is input to the CNN, and the
443 # outputs of each layer, respectively.
444
445 # complete the `forward` function for the `NetQuantized` class.
446 # You will have to add code here to scale the outputs of each layer, and then to clamp the
447 # outputs of each layer to integers between -128 and 127 afterwards.
448
449 # %%
450 from typing import List
451
452 class NetQuantized(nn.Module):
453     def __init__(self, net_with_weights_quantized: nn.Module, bitwidth_A):
454         super(NetQuantized, self).__init__()
455
456         net_init = copy_model(net_with_weights_quantized) # 最初的网络copy 过来,用于初始化
457         量化网络

```

```

447
448     self.conv1 = net_init.conv1
449     self.conv2 = net_init.conv2
450     self.pool = net_init.pool
451     self.conv3 = net_init.conv3
452     self.conv4 = net_init.conv4
453     self.fc1 = net_init.fc1
454     self.fc2 = net_init.fc2
455
456     #self.fc3 = net_init.fc3
457
458     self.activation_border = 2**(bitwidth_A-1)
459     #print(self.activation_border)
460
461     for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.fc1, self.fc2:
462         #逐层遍历,检查量化是否成功
463         def pre_hook(l, x):
464             x = x[0]
465             #异常处理:检查之前的量化是否将所有的数转化进规定的区间
466             if (x < -self.activation_border).any() or (x > (self.activation_border-1))
467
468             .any():
469                 raise Exception("Input to {} layer is out of bounds for an {}-bit
signed integer".format(l.__class__.__name__, bitwidth_A))
470                 if (x != x.round()).any():
471                     raise Exception("Input to {} layer has non-integer values".format(l.
__class__.__name__))
472
473                 layer.register_forward_pre_hook(pre_hook)
474
475             # Calculate the scaling factor for the initial input to the CNN
476             self.input_activations = net_with_weights_quantized.input_activations
477             self.input_scale = NetQuantized.quantize_initial_input(self.input_activations,
self.activation_border)
478             #以上2行计算了input activation 的scale和对应的quantization 版本
479             # Calculate the output scaling factors for all the layers of the CNN
480
481             n_w = []
482             activation_scale= [self.input_scale]
483
484             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.fc1, self.fc2:
485                 # call quantize_activations and add your code here
486
487                 #获取权重缩放因子
488                 n_w.append(layer.weight.scale)
489
490                 # 计算当前层激活的缩放因子
491                 activation_scale.append( NetQuantized.quantize_initial_input(
layer.activations, self.activation_border
) )
492
493             #print(activation_scale)
494
495             n = 0
496             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.fc1, self.fc2:
497                 layer.output_scale = NetQuantized.quantize_activations(n_w[n],
activation_scale[n], activation_scale[n + 1])
498                 n = n + 1
499
500
501             # 注册前向检查钩子
502             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.fc1, self.fc2:
503                 def pre_hook(l, x):

```

```

504         x = x[0]
505         if (x < -self.activation_border).any() or (x > (self.activation_border-1))
.any():
506             raise Exception(f"Input to {l.__class__.__name__} layer is out of
bounds")
507         if (x != x.round()).any():
508             raise Exception(f"Input to {l.__class__.__name__} layer has non-
integer values")
509         layer.register_forward_pre_hook(pre_hook)
510
511     # 计算输入的scale函数
512     @staticmethod
513     def quantize_initial_input(pixels: np.ndarray, activation_border: int):
514         '''
515         Calculate a scaling factor for the images that are input to the first layer of the
CNN.
516
517         Parameters:
518         pixels (ndarray): The values of all the pixels which were part of the input image
during training
519
520         Returns:
521         float: A scaling factor that the input should be multiplied by before being fed
into the first layer.
522             This value does not need to be an 8-bit integer.
523         '''
524
525         sigma3 = int(np.size(pixels)*0.0027+0.5)
526         sigma3_value = pixels[np.argmax(np.abs(pixels), axis=-1)[-sigma3]]
527         max_value = np.max(np.abs(pixels))
528         scale = activation_border/max_value # you can modify the range here, 同样, 这个地
方的scale和论文定义的是和源代码倒数的关系
529         return scale
530
531     # 逐层激活送入量化
532     @staticmethod
533     def quantize_activations( n_w: float, pre_activation_scale: float,
now_activation_scale: float) -> float:
534         '''
535         Calculate a scaling factor to multiply the output of a layer by.
536
537         Parameters:
538         activations (ndarray): The values of all the pixels which have been output by this
layer during training
539         n_w (float): The scale by which the weights of this layer were multiplied as part
of the "quantize_weights" function you wrote earlier
540         n_initial_input (float): The scale by which the initial input to the neural
network was multiplied
541         ns ([[float, float]]): A list of tuples, where each tuple represents the "weight
scale" and "output scale" (in that order) for every preceding layer
542         # 用一个元组记录已经算好的所有网络的weight scale 和 output scale
543         Returns:
544         float: A scaling factor that the layer output should be multiplied by before being
fed into the first layer.
545             This value does not need to be an 8-bit integer.
546         '''
547
548         # ADD YOUR CODE HERE
549         scale = now_activation_scale / (pre_activation_scale * n_w)
550         return scale
551
552     def forward(self, x: torch.Tensor) -> torch.Tensor:

```

```

553         # You can access the output activation scales like this:
554         # fc1_output_scale = self.fc1.output_scale
555
556         # 这里其实是对定点数计算的模拟仿真，因此每层的输出均需要手动乘以scaling factor，并
557         # 限制在规定的表示范围内。
558         # output_scale:不同网络层之间传递整数重新压缩为8bit的scale(在第三周阅读论文中是定
559         # 点浮点数M)
560         # activation_scale:网络层内数据和真实值差距的scale
561         # weight_scale:权重量化值和真实值差距的scale
562         x = x*self.input_scale
563         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
564         -1) # 8bit是-128~127
565
566         x = F.relu(self.conv1(x))
567         x = x*self.conv1.output_scale
568         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
569         -1)
570
571         x = self.pool(F.relu(self.conv2(x)))
572         x = x*self.conv2.output_scale
573         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
574         -1)
575
576         x = F.relu(self.conv3(x))
577         x = x*self.conv3.output_scale
578         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
579         -1)
580
581         x = self.pool(F.relu(self.conv4(x)))
582         x = x*self.conv4.output_scale
583         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
584         -1)
585
586         x = x.view(-1, 32 * 8 * 8)
587         x = F.relu(self.fc1(x))
588         x = x*self.fc1.output_scale
589         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
590         -1)
591
592         #x = F.relu(self.fc2(x))
593         #x = x*self.fc2.output_scale
594         #x = torch.clamp(x.round(), min=-self.activation_border, max=self.
595         activation_border-1)
596
597         x = self.fc2(x)*self.fc2.output_scale
598         return x
599
600
601     # %%
602     #Merge the information from net_q2 and net_q3 together
603     #weight quantization & activation quantization
604     net_init = copy_model(net_q2)
605     net_init.input_activations = deepcopy(net_q3.input_activations)
606     for layer_init, layer_q3 in zip(net_init.children(), net_q3.children()):
607         if isinstance(layer_init, nn.Conv2d) or isinstance(layer_init, nn.Linear):
608             layer_init.activations = deepcopy(layer_q3.activations)
609
610     net_quantized = NetQuantized(net_init, bitwidth_A)
611
612     # %%
613     # Finally, record the accuracy of your network after both weights and activations have

```

```

        been quantized.
606 # If you've done everything right, you should still find almost no accuracy change.
607
608 score = test(net_quantized, testloader)
609 print('Accuracy of the network after quantizing both weights and activations: {}'.format(
        score))
610
611
612 # %%
613 # Question 5: Quantize Biases 以下为选做内容
614
615 # Let us now update our CNN to include a bias in its final layer, *fc3*.
616 # We have already included code to create and train a new CNN called `net_with_bias`.
617
618 # Consider how a bias affects the equation for an unquantized layer:
619
620 #  $W * In + bias = Out$ 
621
622
623
624 # %%
625 ## Question 5.1:
626
627 # Suppose that we again quantized a biased layer with the same scaling factors we used in
        previous questions:  $n_W$ ,  $n_{In}$ , and  $n_{Out}$ .
628 # What would we scale bias by in this case?
629 # Write an equation in your lab report to describe the output of the quantized layer with
        a bias.
630
631 # Create a new network with a bias on *fc3*.
632
633
634 # %%
635 class NetWithBias(nn.Module):
636     def __init__(self):
637         super(NetWithBias, self).__init__()
638         self.conv1 = nn.Conv2d(3, 16, 3, padding='same', bias=False)
639         self.conv2 = nn.Conv2d(16, 16, 3, padding='same', bias=False)
640         self.pool = nn.MaxPool2d(2, 2)
641         self.conv3 = nn.Conv2d(16, 32, 3, padding='same', bias=False)
642         self.conv4 = nn.Conv2d(32, 32, 3, padding='same', bias=False)
643         self.fc1 = nn.Linear(32 * 8 * 8, 128, bias=True)
644         self.fc2 = nn.Linear(128, 10, bias=True)
645
646     def forward(self, x: torch.Tensor) -> torch.Tensor:
647         x = F.relu(self.conv1(x))
648         x = F.relu(self.conv2(x))
649         x = self.pool(x)
650         x = F.relu(self.conv3(x))
651         x = F.relu(self.conv4(x))
652         x = self.pool(x)
653         x = x.view(-1, 32 * 8 * 8)
654         x = F.relu(self.fc1(x))
655         x = self.fc2(x)
656         return x
657
658 net_with_bias = NetWithBias().to(device)
659
660 # %%
661 # Train and test the model
662
663 # %%

```



```

664     train(net_with_bias, trainloader)
665
666     # %%
667     score = test(net_with_bias, testloader)
668     print('Accuracy of the network (with a bias) on the test images: {}'.format(score))
669
670     # %%
671     register_activation_profiling_hooks(net_with_bias)
672     test(net_with_bias, trainloader, max_samples=400)
673     net_with_bias.profile_activations = False
674
675     # %%
676     net_with_bias_with_quantized_weights = copy_model(net_with_bias)
677     quantize_layer_weights(net_with_bias_with_quantized_weights, bitwidth_W)
678
679     score = test(net_with_bias_with_quantized_weights, testloader)
680     print('Accuracy of the network on the test images after all the weights are quantized but
        the bias isn\'t: {}'.format(score))
681
682     # %%
683     # Fill in the `quantized_bias` function in the `NetQuantizedWithBias` class.
684     # This function is meant to quantize the bias on the final layer of the CNN.
685     # Keep in mind that biases are typically quantized to 32-bits, so your bias values do not
        all have to be between -128 and 127 (though 32-bits is a bit conservative).
686
687     # %%
688     # slightly clearer bias bounds (32b signed integer)
689     #bias 量化后的最大值、最小值
690     MIN_32B_SINT = -(2**31)
691     MAX_32B_SINT = (2**31) - 1
692
693     # %%
694     class NetQuantizedWithBias(NetQuantized):
695         def __init__(self, net_with_weights_quantized: nn.Module):
696             super(NetQuantizedWithBias, self).__init__(net_with_weights_quantized, bitwidth_A)
697
698             preceding_scales_fc2 = [(layer.weight.scale, layer.output_scale) for layer in self
        .children() if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear)][:-1]
699             preceding_scales_fc1 = preceding_scales_fc2[0:4]
700             #print(preceding_scales_fc1)
701
702             self.fc1.bias.data = NetQuantizedWithBias.quantized_bias(
703                 self.fc1.bias.data,
704                 self.fc1.weight.scale,
705                 self.input_scale,
706                 preceding_scales_fc1
707             )
708             #print(self.fc1.bias.data)
709
710             self.fc2.bias.data = NetQuantizedWithBias.quantized_bias(
711                 self.fc2.bias.data,
712                 self.fc2.weight.scale,
713                 self.input_scale,
714                 preceding_scales_fc2
715             )
716             #print(self.fc2.bias.data)
717
718             if (self.fc1.bias.data < MIN_32B_SINT).any() or (self.fc1.bias.data > MAX_32B_SINT
        ).any():
719                 raise Exception("Bias has values which are out of bounds for an 32-bit signed
        integer")
720             if (self.fc1.bias.data != self.fc1.bias.data.round()).any():

```

```

721         raise Exception("Bias has non-integer values")
722         if (self.fc2.bias.data < MIN_32B_SINT).any() or (self.fc2.bias.data > MAX_32B_SINT
723             ).any():
724             raise Exception("Bias has values which are out of bounds for an 32-bit signed
725                 integer")
726             if (self.fc2.bias.data != self.fc2.bias.data.round()).any():
727                 raise Exception("Bias has non-integer values")
728
729     @staticmethod
730     def quantized_bias(bias: torch.Tensor, n_w: float, n_initial_input: float, ns: List[
731         Tuple[float, float]]) -> torch.Tensor:
732         """
733         Quantize the bias so that all values are integers between MIN_32B_SINT and
734         MAX_32B_SINT.
735
736         Parameters:
737         bias (Tensor): The floating point values of the bias
738         n_w (float): The scale by which the weights of this layer were multiplied
739         n_initial_input (float): The scale by which the initial input to the neural
740         network was multiplied
741         ns ((float, float)): A list of tuples, where each tuple represents the "weight
742         scale" and "output scale" (in that order) for every preceding layer
743
744         Returns:
745         Tensor: The bias in quantized form, where every value is an integer between
746         MIN_32B_SINT and MAX_32B_SINT.
747         The "dtype" will still be "float", but the values themselves should all be
748         integers.
749         """
750         # ADD YOUR CODE HERE
751
752         #print(bias)
753         # 计算累积缩放因子: 初始输入缩放 * 所有前驱层的输出缩放
754
755         scale = n_initial_input
756         for weight_scale, output_scale in ns:
757             scale *= output_scale
758             scale *= weight_scale
759
760         # 总缩放因子 = 累积输入缩放 * 当前层权重缩放
761         scale = scale * n_w
762         print(scale)
763
764         # 量化偏置: bias * scale → 四舍五入 → 截断至32位整数范围
765         quan_bias = (bias * scale).round()
766         quan_bias = torch.clamp(quan_bias, min=MIN_32B_SINT, max=MAX_32B_SINT)
767         return quan_bias
768
769 # %%
770 net_quantized_with_bias = NetQuantizedWithBias(net_with_bias_with_quantized_weights)
771
772 # %%
773 ## Question 5.3:
774
775 # What is your accuracy before and after quantizing CNN with the bias?
776 # The accuracy change should ideally be negligible.

```

```

775 # %%
776 score = test(net_quantized_with_bias, testloader)
777 print('Accuracy of the network on the test images after all the weights and the bias are
quantized: {}'.format(score))

```

Listing 7: lab1_PTQ_deepernet.ipynb

```

1 # %%
2 # 本次实验包括后量化的过程，主要在于学习如何对DNN的权重（weight）和激活层（activation）进
  行量化。
3
4 # %%
5 # 引用外部软件包
6 import torch # pytorch包
7 import torchvision # torchvision是pytorch官方提供的工具，包含很多常用数据集的封装等
8 import torchvision.transforms as transforms
9 import torch.nn as nn # nn 和下面的F均包含了主要的神经网络模块，包括卷积层、全连接层、池化
  层等等
10 import torch.nn.functional as F
11 import torch.optim as optim # 优化器
12
13 # %%
14 # 确保device为gpu，否则会花费过长时间。
15 if torch.cuda.is_available():
16     torch.cuda.set_device(0) # 0为选择显卡id
17     device = "cuda"
18     print('using cuda')
19 else:
20     device = torch.device('cpu')
21     print('using cpu')
22
23 # 设置量化位宽
24 bitwidth_A = 8
25 bitwidth_W = 8
26
27 # %%
28 # Load training and test data from the CIFAR10 dataset.
29 # torchvision是pytorch官方集成的工具库，包括学界常用的数据集和数据预处理方式。
30 transform = transforms.Compose(
31     [transforms.ToTensor(), # 将数据归一化，并转换成tensor的形式
32      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))] # 预处理数据的方式，
33     normalize是将图像进行规则化。
34
35 trainset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset', train=True,
36                                         download=True, transform=transform) #定义了训练
  集，path是训练数据所在地址，指定位置自动下载。
37 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
38                                           shuffle=True, num_workers=2) #定义数据迭代器，
39 batchsize为一次迭代训练，送进去图像的数量（可以尝试调整，看看会发生什么）。shuffle表示要不
  要打乱训练图像的排序。
40
41 testset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset', train=False,
42                                         download=True, transform=transform)
43 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
44                                         shuffle=False, num_workers=2)
45
46 # %%
47 # Define a CNN that classifies CIFAR images.
48 # The Resnet has the following architecture:

```

```

48 # **Layer** |      **Type**      | **Input Shape** | **Output Shape** | **Activation**
49 #  conv1   | Convolutional   | 3x32x32         | 64x32x32         | ReLU
50 #  pool1   | Max pool        | 64x32x32         | 64x16x16         | None
51 #  conv2   | Convolutional   | 64x16x16         | 64x16x16         | ReLU
52 #  conv3   | Convolutional   | 64x16x16         | 64x16x16         | ReLU
53 #  conv4   | Convolutional   | 64x16x16         | 64x16x16         | ReLU
54 #  conv5   | Convolutional   | 64x16x16         | 128x8x8          | ReLU
55 #  conv6   | Convolutional   | 128x8x8          | 128x8x8          | ReLU
56 #  conv7   | Convolutional   | 128x8x8          | 128x8x8          | ReLU
57 #  conv8   | Convolutional   | 64x16x16         | 128x8x8          | ReLU
58 #  pool2   | Avg pool        | 128x8x8          | 128x4x4          | None
59 #  fc1     | Fully-connected | 128x4x4          | 128              | ReLU
60 #  fc2     | Fully-connected | 128              | 10               | ReLU
61
62 # None of the layers in the network have a bias associated with them.
63 # This makes them easier to quantize.
64 # Towards the end of this assignment, we will add biases to the final layer and quantize
  it as well.
65
66 # %%
67 # 网络定义, conv2d是卷积层, linear是全连接层。forward下定义了计算图, 显示送进来的图像(x)
  会怎么处理。
68 class Net(nn.Module):      #网络结构:8层卷积层、2层池化层、2层全连接层, 留意这些参数的内涵
69     def __init__(self):
70         super(Net, self).__init__()
71         self.conv1 = nn.Conv2d(3, 64, 3, padding='same', bias=False)
72         self.bn1 = nn.BatchNorm2d(64, momentum= 0.1)
73         self.pool = nn.MaxPool2d(2, 2) # run after each conv (hence the 3x3 FC layer)    16
74
75         self.conv2 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
76         self.bn2 = nn.BatchNorm2d(64, momentum= 0.1)
77         self.conv3 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
78         self.bn3 = nn.BatchNorm2d(64, momentum= 0.1)
79         self.conv4 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
80         self.bn4 = nn.BatchNorm2d(64, momentum= 0.1)
81         self.conv5 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
82         self.bn5 = nn.BatchNorm2d(64, momentum= 0.1)
83         self.conv6 = nn.Conv2d(64, 128, 3, padding=1, stride = 2, bias=False)      #8x8
84         self.bn6 = nn.BatchNorm2d(128, momentum= 0.1)
85         self.conv7 = nn.Conv2d(128, 128, 3, padding='same', bias=False)
86         self.bn7 = nn.BatchNorm2d(128, momentum= 0.1)
87         self.conv8 = nn.Conv2d(64, 128, 1, stride = 2, bias=False)
88         self.bn8 = nn.BatchNorm2d(128, momentum= 0.1)
89         self.pool2 = nn.AvgPool2d(2,2)
90         self.fc1 = nn.Linear(128 * 4 * 4, 128, bias=False)
91         self.fc2 = nn.Linear(128, 10, bias=False)
92
93         #定义前向传播,将tensor数据送入网络之后的计算图
94     def forward(self, x: torch.Tensor) -> torch.Tensor:
95         x = F.relu(self.conv1(x))
96         x = self.bn1(x)
97         x = self.pool(x)
98
99         temp = x
100         x = F.relu(self.conv2(x))
101         x = self.bn2(x)
102         x = F.relu(self.conv3(x))
103         x = self.bn3(x)
104         x = x + temp
105
106         temp = x
107         x = F.relu(self.conv4(x))
108         x = self.bn4(x)

```

```

107         x = F.relu(self.conv5(x))
108         x = self.bn5(x)
109         x = x + temp
110
111         temp = x
112         x = F.relu(self.conv6(x))
113         x = self.bn6(x)
114         x = F.relu(self.conv7(x))
115         x = self.bn7(x)
116         temp = F.relu(self.conv8(temp))
117         temp = self.bn8(temp)
118         x = x + temp
119
120         x = self.pool2(x)
121
122         x = x.view(-1, 128 * 4 * 4)
123         x = F.relu(self.fc1(x))
124         x = self.fc2(x)
125
126         return x
127
128 net = Net().to(device)
129 # 到这里，我们定义好了网络和数据集的使用方式
130
131 # %%
132 from torch.utils.data import DataLoader
133 # 将训练集的dataloader送入网络进行训练，每次读取batchsize数量的数据
134 def train(model: nn.Module, dataloader: DataLoader):
135     criterion = nn.CrossEntropyLoss() # loss function (交叉熵损失函数)
136     optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) #optimizer, 使用SGD
137     # 优化器训练，第一个参数表示model的所有参数都会参与训练，学习率为0.001，动量为0.9
138
139     for epoch in range(3): # loop over the dataset multiple times。epoch指遍历所有训练集
140         # 图像的个数。这里遍历所有训练图像两次
141
142         running_loss = 0.0
143         for i, data in enumerate(dataloader, 0): # dataloader是数据迭代器，从这里每次读出
144             # batchsize数量的数据。
145             # get the inputs; data is a list of [inputs, labels]
146             inputs, labels = data
147
148             inputs = inputs.to(device)
149             labels = labels.to(device)
150
151             # zero the parameter gradients
152             optimizer.zero_grad()
153
154             # forward + backward + optimize
155             outputs = model(inputs) # 将数据送进网络，得到结果
156             loss = criterion(outputs, labels) # 计算结果和标签的交叉熵损失函数
157             loss.backward() # 基于上述损失函数求解梯度
158             optimizer.step() # 更新参数
159
160             # print statistics
161             running_loss += loss.item() # 将loss的值取出来，记录其变化
162             if i % 2000 == 0: # print every 2000 mini-batches
163                 print('[%d, %5d] loss: %.3f' %
164                       (epoch + 1, i + 1, running_loss / 2000))
165                 running_loss = 0.0
166
167 print('Finished Training')

```

```

166 def test(model: nn.Module, dataloader: DataLoader, max_samples=None) -> float:
167     correct = 0
168     total = 0
169     n_inferences = 0
170
171     with torch.no_grad(): # 测试时不用求解梯度，因此可以设置不计算梯度的模式
172         for data in dataloader:
173             images, labels = data
174
175             images = images.to(device)
176             labels = labels.to(device)
177
178             outputs = model(images) # get 1 batch worth of image predictions (i.e. 4
179             # predictions of 10 each)
180             other, predicted = torch.max(outputs.data, 1) # other == values, predicted ==
181             # indicies
182             total += labels.size(0)
183             correct += (predicted == labels).sum().item()
184
185             if max_samples:
186                 n_inferences += images.shape[0]
187                 if n_inferences > max_samples:
188                     break
189
190             return 100 * correct / total
191
192 # %%
193 train(net, trainloader) # 进行训练，将网络定义和数据迭代器传给train函数。
194
195 # %%
196 # Now that the CNN has been trained, let's test it on our test dataset.
197 score = test(net, testloader)
198 print('Accuracy of the network on the test images: {}'.format(score))
199
200 # %%
201 # Define a convenience function which we use to copy CNN's.
202 from copy import deepcopy
203
204 def copy_model(model: nn.Module) -> nn.Module:
205     result = deepcopy(model)
206
207     # Copy over the extra metadata we've collected which copy.deepcopy doesn't capture
208     if hasattr(model, 'input_activations'):
209         result.input_activations = deepcopy(model.input_activations)
210
211     for result_layer, original_layer in zip(result.children(), model.children()):
212         if isinstance(result_layer, nn.Conv2d) or isinstance(result_layer, nn.Linear):
213             if hasattr(original_layer, 'weight', 'scale'):
214                 result_layer.weight.scale = deepcopy(original_layer.weight.scale)
215             if hasattr(original_layer, 'activations'):
216                 result_layer.activations = deepcopy(original_layer.activations)
217             if hasattr(original_layer, 'output_scale'):
218                 result_layer.output_scale = deepcopy(original_layer.output_scale)
219
220     return result
221
222 # %%
223 # Question 1: Visualize Weights
224
225 # %%
226 ## Question 1.1:

```

```

226 import matplotlib.pyplot as plt
227 import numpy as np
228
229 name = ['conv1', 'conv2', 'conv3', 'conv4', 'conv5', 'conv6', 'conv7', 'conv8', 'fc1', '
      fc2']
230 n = 0
231
232 for layer in net.children(): # 遍历类的成员
233     if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
234         weight = layer.weight.data.cpu().view(-1).numpy()
235         print(name[n] + ": the range is [{:.5f}, {:.5f}].format(weight.min(), weight.max
      ()))
236
237         # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
238         sigma3 = int(np.size(weight)*0.0027+0.5)
239         sigma3_value = np.abs(weight)[np.argpartition(np.abs(weight), -sigma3)[-sigma3]]
240         print(name[n] + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value
      , weight.min()), min(sigma3_value, weight.max()))
241
242         plt.hist(weight, bins=50, facecolor="blue", edgecolor="black", alpha=0.7) # 画直方
      图
243         plt.xlabel("range")
244         plt.ylabel("frequency")
245         plt.title(name[n])
246         plt.show()
247         n += 1
248
249 # You can get a flattened vector of the weights of fc1 like this:
250 # fc1_weights = net.fc1.weight.data.cpu().view(-1)
251
252 # %%
253 ## Question 1.2: Explain which range you would prefer to use if you were to quantize each
      layer's weights and wanted to strike a balance between the range of values that could be
      expressed, and your precision.
254
255 # %%
256 # Question 2: Quantize Weights 首先我们把原来网络的参数copy到net_q2。
257
258 net_q2 = copy_model(net)
259
260 # %%
261 ## Question 2.1:
262
263 from typing import Tuple
264 # 确定scale进行量化
265 def quantized_weights(weights: torch.Tensor, bitwidth_W):
266     '''
267     Quantize the weights so that all values are integers between -128 and 127.
268     You may want to use the total range, 3-sigma range, or some other range when
269     deciding just what factors to scale the float32 values by.
270
271     Parameters:
272     weights (Tensor): The unquantized weights
273     #输入数据是尚未量化过的权重
274
275     Returns:
276     #返回数据是一个元组,(quantization result , scale )
277     (Tensor, float): A tuple with the following elements:
278         * The weights in quantized form, where every value is an integer
      between -128 and 127.
279         The "dtype" will still be "float", but the values themselves
      should all be integers.

```

```

280         * The scaling factor that your weights were multiplied by.
281         This value does not need to be an 8-bit integer.
282
283
284     # modify the following code to try other range
285
286     W_border = 2**(bitwidth_W-1)
287     w = weights.data.cpu().view(-1).numpy() # 这里将tensor转入numpy只是为了之后的计算
288     with open('test.txt', 'w') as f:
289         for item in w: # 打开test.txt 如果文件不存在, 创建该文件。
290             f.write(str(item)+'\n') # 把变量var写入test.txt。这里var必须是str格式, 如果不是, 则可以转一下。
291
292     sigma3 = int(np.size(w)*0.0027+0.5)
293     sigma3_value = np.abs(w)[np.argmax(np.abs(w), -sigma3)[-sigma3]]
294     max_value = np.nanmax(np.abs(w))
295     #print(max_value)
296     weights = weights/sigma3_value*W_border
297     quan_weight = weights.round() # 转化为值最接近的整数
298     #print(quan_weight)
299
300     return torch.clamp(quan_weight, min=-W_border, max=W_border-1), W_border/sigma3_value
301     # clamp是切顶函数, 会把超过阈值的值自动替换成阈值
302
303     # 我认为原来对scale的定义有问题, 应该是反过来才对
304
305 # %%
306 def quantize_layer_weights(model: nn.Module, bitwidth_W):
307     for layer in model.children():
308         if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear):
309             q_layer_data, scale = quantized_weights(layer.weight.data, bitwidth_W)
310             q_layer_data = q_layer_data.to(device)
311
312             layer.weight.data = q_layer_data
313             layer.weight.scale = scale
314             #print(layer.weight.data) # 一共有8个卷积层和2个全连接层, 因此会有10个scale
315
316             if (q_layer_data < -2**(bitwidth_W-1)).any() or (q_layer_data > (2**(
317 bitwidth_W-1)-1)).any():
318                 raise Exception("Quantized weights of {} layer include values out of
319 bounds for an 8-bit signed integer".format(layer.__class__.__name__))
320             if (q_layer_data != q_layer_data.round()).any():
321                 raise Exception("Quantized weights of {} layer include non-integer values"
322 .format(layer.__class__.__name__))
323
324 quantize_layer_weights(net_q2, bitwidth_W)
325
326 # %%
327 ## Question 2.2:
328
329 #Record the accuracy change of the network after quantizing its weights. If you've done
330 everything correctly, the accuracy change should be negligible.
331
332 score = test(net_q2, testloader)
333 print('Accuracy of the network after quantizing all weights: {}'.format(score))
334
335 # %%
336 # Question 3: Visualize Activations
337
338 # Now that we have quantized the weights of the CNN, we must also quantize the activations

```



```

    (inputs and outputs to layers) traveling through it.
335 # But before doing so, let's analyze what values the activations take when travelling
    through the network.
336
337 # We provide convenience code which will record the values of every pixel of the outputs
    and inputs travelling through the neural network.
338 # (This is the initial CNN, where not even the weights had yet been quantized).
339 # We then profile these values when running on a subset of the training set.
340
341
342 # %%
343 # hook, 顾名思义, 其作用是把网络推理时的中间变量勾出来。通过hook函数, 我们可以统计中间变量
    的范围。
344 """
345 pytorch中的hook机制:
346 Pytorch会自动舍弃图计算中间结果,用hook以获取某些变量的中间结果.要及时删除,避免增加运行负载
347
348 Tensor级别:
349 register_hook(hook)->Tensor:为Tensor注册一个backward hook 获取变量梯度
350 hook遵循格式:hook(grad)->Tensor
351 nn.Module对象级别:
352 register_forward_hook(hook)
353 此时hook可以修改input和output却不影响forward结果,典型场景是提取中间层的输出特征,又不希望修
    改原有模型定义文件
354 格式为hook(layer,input,output)
355 """
356 def register_activation_profiling_hooks(model: Net):
357     model.input_activations = np.empty(0)
358     model.conv1.activations = np.empty(0)
359     model.conv2.activations = np.empty(0)
360     model.conv3.activations = np.empty(0)
361     model.conv4.activations = np.empty(0)
362     model.conv5.activations = np.empty(0)
363     model.conv6.activations = np.empty(0)
364     model.conv7.activations = np.empty(0)
365     model.conv8.activations = np.empty(0)
366     model.bn1.activations = np.empty(0)
367     model.bn2.activations = np.empty(0)
368     model.bn3.activations = np.empty(0)
369     model.bn4.activations = np.empty(0)
370     model.bn5.activations = np.empty(0)
371     model.bn6.activations = np.empty(0)
372     model.bn7.activations = np.empty(0)
373     model.bn8.activations = np.empty(0)
374     model.fc1.activations = np.empty(0)
375     model.fc2.activations = np.empty(0)
376
377     model.profile_activations = True
378
379     def conv1_activations_hook(layer, x, y):
380         if model.profile_activations:
381             model.input_activations = np.append(model.input_activations, x[0].cpu().view
    (-1))
382             model.conv1.register_forward_hook(conv1_activations_hook)#寄存器暂时存下传递到下一层的
    activatio,下同
383
384
385     def conv2_activations_hook(layer, x, y):
386         if model.profile_activations:
387             model.conv1.activations = np.append(model.conv1.activations, x[0].cpu().view
    (-1))
388             model.conv2.register_forward_hook(conv2_activations_hook)

```

```

389
390     def conv3_activations_hook(layer, x, y):
391         if model.profile_activations:
392             model.conv2.activations = np.append(model.conv2.activations, x[0].cpu().view
(-1))
393             model.conv3.register_forward_hook(conv3_activations_hook)
394
395     def conv4_activations_hook(layer, x, y):
396         if model.profile_activations:
397             model.conv3.activations = np.append(model.conv3.activations, x[0].cpu().view
(-1))
398             model.conv4.register_forward_hook(conv4_activations_hook)
399
400     def conv5_activations_hook(layer, x, y):
401         if model.profile_activations:
402             model.conv4.activations = np.append(model.conv4.activations, x[0].cpu().view
(-1))
403             model.conv5.register_forward_hook(conv5_activations_hook)#寄存器暂时存下传递到下一层的
activatio,下同
404
405     def conv6_activations_hook(layer, x, y):
406         if model.profile_activations:
407             model.conv5.activations = np.append(model.conv5.activations, x[0].cpu().view
(-1))
408             model.conv6.register_forward_hook(conv6_activations_hook)
409
410     def conv7_activations_hook(layer, x, y):
411         if model.profile_activations:
412             model.conv6.activations = np.append(model.conv6.activations, x[0].cpu().view
(-1))
413             model.conv7.register_forward_hook(conv7_activations_hook)
414
415     def conv8_activations_hook(layer, x, y):
416         if model.profile_activations:
417             model.conv7.activations = np.append(model.conv7.activations, x[0].cpu().view
(-1))
418             model.conv8.register_forward_hook(conv8_activations_hook)
419
420     def fc1_activations_hook(layer, x, y):
421         if model.profile_activations:
422             model.conv8.activations = np.append(model.conv8.activations, x[0].cpu().view
(-1))
423             model.fc1.register_forward_hook(fc1_activations_hook)
424
425
426     def fc2_activations_hook(layer, x, y):
427         if model.profile_activations:
428             model.fc1.activations = np.append(model.fc1.activations, x[0].cpu().view(-1))
429             model.fc2.activations = np.append(model.fc2.activations, y[0].cpu().view(-1))
430             model.fc2.register_forward_hook(fc2_activations_hook)
431
432
433     # %%
434     # Use the unquantized model to profile input and output activations on a subset of the
training dataset.
435
436     net_q3 = copy_model(net)
437     register_activation_profiling_hooks(net_q3)#把各个中间层都记录下来
438
439     # Run through the training dataset again while profiling the input and output activations
this time
440     # We don't actually have to perform gradient descent for this, so we can use the "test"

```

```

function
441 test(net_q3, trainloader, max_samples=400) # 跑400次, 统计中间变量的数值范围
442 net_q3.profile_activations = False
443
444 # %%
445 ## Question 3.1:
446
447 # Plot histograms of the input images and the outputs of every convolutional and fully-
448 # connected layer.
449 # Record any observations you make about the distribution of the values.
450 # ADD YOUR CODE HERE
451 Activation_border = 2*(bitwidth_A-1)
452 input_activations = net_q3.input_activations # 输入数据
453 conv1_output_activations = net_q3.bn1.activations # 第一层卷积层的输出数据
454 conv2_output_activations = net_q3.bn2.activations # 第二层卷积层的输出数据
455 conv3_output_activations = net_q3.bn3.activations # 第三层卷积层的输出数据
456 conv4_output_activations = net_q3.bn4.activations # 第四层卷积层的输出数据
457 conv5_output_activations = net_q3.bn5.activations # 第一层卷积层的输出数据
458 conv6_output_activations = net_q3.bn6.activations # 第二层卷积层的输出数据
459 conv7_output_activations = net_q3.bn7.activations # 第三层卷积层的输出数据
460 conv8_output_activations = net_q3.bn8.activations # 第四层卷积层的输出数据
461 fc1_output_activations = net_q3.fc1.activations # 同上类推
462 fc2_output_activations = net_q3.fc2.activations
463
464 # to plot distributions of activations
465 # Plot histograms of the following variables, and calculate their ranges and 3-sigma
466 # ranges:
467 # 拼接上面的数据成一个group, 方便循环遍历
468 activation_groups = [
469     (input_activations, "input_activations"),
470     (conv1_output_activations, "conv1_output_activations"),
471     (conv2_output_activations, "conv2_output_activations"),
472     (conv3_output_activations, "conv3_output_activations"),
473     (conv4_output_activations, "conv4_output_activations"),
474     (conv5_output_activations, "conv5_output_activations"),
475     (conv6_output_activations, "conv6_output_activations"),
476     (conv7_output_activations, "conv7_output_activations"),
477     (conv8_output_activations, "conv8_output_activations"),
478     (fc1_output_activations, "fc1_output_activations"),
479     (fc2_output_activations, "fc2_output_activations"),
480 ]
481
482 for activation_data, name in activation_groups: # 遍历类的成员
483     print(name + ": the range is [{:.5f}, {:.5f}].format(activation_data.min(),
484         activation_data.max()))
485
486     # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
487     sigma3 = int(np.size(activation_data)*0.0027+0.5)
488     sigma3_value = np.abs(activation_data)[np.argsort(np.abs(activation_data), -
489         sigma3)[-sigma3]]
490     print(name + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value,
491         activation_data.min()), min(sigma3_value, activation_data.max()))
492
493     plt.hist(activation_data, bins=50, facecolor="green", edgecolor="black", alpha=0.7) #
494     # 画直方图
495     plt.xlabel("range")
496     plt.ylabel("frequency")
497     plt.title(name+"_distribution")
498     plt.show()
499
500 # You can get a flattened vector of the weights of fc1 like this:

```

```

496 # fc1_weights = net.fc1.weight.data.cpu().view(-1)
497
498
499 # %%
500 ## Question 3.2:
501
502 # Additionally, record the range of the values, as well as their 3-sigma range (the
503 # difference between  $\mu + 3\sigma$  and  $\mu - 3\sigma$ ).
504 # For which layers is the 3-sigma range larger or smaller than the actual range?
505 # Then explain which range you would prefer to use if you were to quantize each layer's
506 # weights and wanted to strike a balance between the range of values that could be expressed
507 # , and your precision.
508 # Remember that you are plotting the activations *after* activation functions like ReLU
509 # have been applied, which means that you should not be worried if you find that your plots
510 # are asymmetric.
511
512 # %%
513 # Question 4: Quantize Activations
514
515 # Complete the `quantize_initial_input` and `quantize_activations` functions which
516 # calculate the scaling factors for the initial image which is input to the CNN, and the
517 # outputs of each layer, respectively.
518
519 # complete the `forward` function for the `NetQuantized` class.
520 # You will have to add code here to scale the outputs of each layer, and then to clamp the
521 # outputs of each layer to integers between -128 and 127 afterwards.
522
523 # %%
524 from typing import List
525
526 class NetQuantized(nn.Module):
527     def __init__(self, net_with_weights_quantized: nn.Module, bitwidth_A):
528         super(NetQuantized, self).__init__()
529
530         net_init = copy_model(net_with_weights_quantized) # 最初的网络copy 过来,用于初始化
531         量化网络
532
533         self.conv1 = net_init.conv1
534         self.bn1 = net_init.bn1
535         self.conv2 = net_init.conv2
536         self.bn2 = net_init.bn2
537         self.pool = net_init.pool
538         self.conv3 = net_init.conv3
539         self.bn3 = net_init.bn3
540         self.conv4 = net_init.conv4
541         self.bn4 = net_init.bn4
542         self.conv5 = net_init.conv5
543         self.bn5 = net_init.bn5
544         self.conv6 = net_init.conv6
545         self.bn6 = net_init.bn6
546         self.conv7 = net_init.conv7
547         self.bn7 = net_init.bn7
548         self.conv8 = net_init.conv8
549         self.bn8 = net_init.bn8
550         self.pool2 = net_init.pool2
551         self.fc1 = net_init.fc1
552         self.fc2 = net_init.fc2
553
554
555         self.activation_border = 2**(bitwidth_A-1)
556         #print(self.activation_border)
557

```

```

549         for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.conv5, self.
conv6, self.conv7, self.conv8, self.fc1, self.fc2:
550             #逐层遍历,检查量化是否成功
551             def pre_hook(l, x):
552                 x = x[0]
553                 #异常处理:检查之前的量化是否将所有的数转化进规定的区间
554                 if (x < -self.activation_border).any() or (x > (self.activation_border-1))
.any():
555                     raise Exception("Input to {} layer is out of bounds for an {}-bit
signed integer".format(l.__class__.__name__, bitwidth_A))
556                     if (x != x.round()).any():
557                         raise Exception("Input to {} layer has non-integer values".format(l.
__class__.__name__))
558
559                 layer.register_forward_pre_hook(pre_hook)
560
561             # Calculate the scaling factor for the initial input to the CNN
562             self.input_activations = net_with_weights_quantized.input_activations
563             self.input_scale = NetQuantized.quantize_initial_input(self.input_activations,
self.activation_border)
564             #以上2行计算了input activation 的scale和对应的quantization 版本
565             # Calculate the output scaling factors for all the layers of the CNN
566
567             n_w = []
568             activation_scale= [self.input_scale]
569
570             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.conv5, self.
conv6, self.conv7, self.conv8, self.fc1, self.fc2:
571                 # call quantize_activations and add your code here
572
573                 #获取权重缩放因子
574                 n_w.append(layer.weight.scale)
575
576                 # 计算当前层激活的缩放因子
577                 activation_scale.append( NetQuantized.quantize_initial_input(
layer.activations, self.activation_border
578                 ) )
579
580             #print(activation_scale)
581
582             n = 0
583             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.conv5, self.
conv6, self.conv7, self.conv8, self.fc1, self.fc2:
584                 layer.output_scale = NetQuantized.quantize_activations(n_w[n],
activation_scale[n], activation_scale[n + 1])
585                 n = n + 1
586
587
588             # 注册前向检查钩子
589             for layer in self.conv1, self.conv2, self.conv3, self.conv4, self.conv5, self.
conv6, self.conv7, self.conv8, self.fc1, self.fc2:
590                 def pre_hook(l, x):
591                     x = x[0]
592                     if (x < -self.activation_border).any() or (x > (self.activation_border-1))
.any():
593                         raise Exception(f"Input to {l.__class__.__name__} layer is out of
bounds")
594                         if (x != x.round()).any():
595                             raise Exception(f"Input to {l.__class__.__name__} layer has non-
integer values")
596
597                 layer.register_forward_pre_hook(pre_hook)
598

```

```

599     #计算输入的scale函数
600     @staticmethod
601     def quantize_initial_input(pixels: np.ndarray, activation_border:int):
602         '''
603         Calculate a scaling factor for the images that are input to the first layer of the
        CNN.
604
605         Parameters:
606         pixels (ndarray): The values of all the pixels which were part of the input image
        during training
607
608         Returns:
609         float: A scaling factor that the input should be multiplied by before being fed
        into the first layer.
610             This value does not need to be an 8-bit integer.
611         '''
612
613         sigma3 = int(np.size(pixels)*0.0027+0.5)
614         sigma3_value = pixels[np.argpartition(pixels,-sigma3)[-sigma3]]
615         max_value = np.max(np.abs(pixels))
616         scale = activation_border/max_value # you can modify the range here, 同样, 这个地
        方的scale和论文定义的是和源代码倒数的关系
617         return scale
618
619     # 逐层激活送入量化
620     @staticmethod
621     def quantize_activations( n_w: float, pre_activation_scale: float,
        now_activation_scale: float) -> float:
622         '''
623         Calculate a scaling factor to multiply the output of a layer by.
624
625         Parameters:
626         activations (ndarray): The values of all the pixels which have been output by this
        layer during training
627         n_w (float): The scale by which the weights of this layer were multiplied as part
        of the "quantize_weights" function you wrote earlier
628         n_initial_input (float): The scale by which the initial input to the neural
        network was multiplied
629         ns ([[float, float]]): A list of tuples, where each tuple represents the "weight
        scale" and "output scale" (in that order) for every preceding layer
630         #用一个元组记录已经算好的所有网络的weight scale 和 output scale
631         Returns:
632         float: A scaling factor that the layer output should be multiplied by before being
        fed into the first layer.
633             This value does not need to be an 8-bit integer.
634         '''
635
636         # ADD YOUR CODE HERE
637         scale = now_activation_scale / (pre_activation_scale * n_w)
638         return scale
639
640     def forward(self, x: torch.Tensor) -> torch.Tensor:
641         # You can access the output activation scales like this:
642         # fc1_output_scale = self.fc1.output_scale
643
644         # 这里其实是对定点数计算的模拟仿真, 因此每层的输出均需要手动乘以scaling factor, 并
        限制在规定的表示范围内。
645         # output_scale:不同网络层之间传递整数重新压缩为8bit的scale(在第三周阅读论文中是定
        点浮点数M)
646         # activation_scale:网络层内数据和真实值差距的scale
647         # weight_scale:权重量化值和真实值差距的scale
648         x = x*self.input_scale

```

```

649         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1) # 8bit是-128-127
650
651         x = F.relu(self.conv1(x))
652         x = x*self.conv1.output_scale
653         x = self.bn1(x)
654         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
655
656         x = self.pool(x)
657
658         temp = x
659
660         x = F.relu(self.conv2(x))
661         x = x*self.conv2.output_scale
662         x = self.bn2(x)
663         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
664
665
666         x = F.relu(self.conv3(x))
667         x = x*self.conv3.output_scale
668         x = self.bn3(x)
669         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
670         temp = torch.clamp(temp.round(), min=-self.activation_border, max=self.
activation_border-1)
671         x = x + temp
672
673         temp = x
674
675         x = F.relu(self.conv4(x))
676         x = x*self.conv4.output_scale
677         x = self.bn4(x)
678         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
679
680         x = F.relu(self.conv5(x))
681         x = x*self.conv5.output_scale
682         x = self.bn5(x)
683         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
684         temp = torch.clamp(temp.round(), min=-self.activation_border, max=self.
activation_border-1)
685         x = x + temp
686
687         temp = x
688
689         x = F.relu(self.conv6(x))
690         x = x*self.conv6.output_scale
691         x = self.bn6(x)
692         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
693
694         x = F.relu(self.conv7(x))
695         x = x*self.conv7.output_scale
696         x = self.bn7(x)
697         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
698         temp = torch.clamp(temp.round(), min=-self.activation_border, max=self.
activation_border-1)
699

```

```

700         temp = F.relu(self.conv8(temp))
701         temp = temp*self.conv8.output_scale
702         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
703         temp = self.bn8(temp)
704         temp = torch.clamp(temp.round(), min=-self.activation_border, max=self.
activation_border-1)
705         x = x + temp
706
707         x = self.pool2(x)
708         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
709
710         x = x.view(-1, 128 * 4 * 4)
711         x = F.relu(self.fc1(x))
712         x = x*self.fc1.output_scale
713         x = torch.clamp(x.round(), min=-self.activation_border, max=self.activation_border
-1)
714
715         x = self.fc2(x)*self.fc2.output_scale
716         return x
717
718
719 # %%
720 #Merge the information from net_q2 and net_q3 together
721 #weight quantization & activation quantization
722 net_init = copy_model(net_q2)
723 net_init.input_activations = deepcopy(net_q3.input_activations)
724 for layer_init, layer_q3 in zip(net_init.children(), net_q3.children()):
725     if isinstance(layer_init, nn.Conv2d) or isinstance(layer_init, nn.Linear):
726         layer_init.activations = deepcopy(layer_q3.activations)
727
728 net_quantized = NetQuantized(net_init, bitwidth_A)
729
730
731 # %%
732 # Finally, record the accuracy of your network after both weights and activations have
been quantized.
733 # If you've done everything right, you should still find almost no accuracy change.
734
735 score = test(net_quantized, testloader)
736 print('Accuracy of the network after quantizing both weights and activations: {}'.format(
score))
737
738
739 # %%
740 # Question 5: Quantize Biases 以下为选做内容
741
742 # Let us now update our CNN to include a bias in its final layer, *fc3*.
743 # We have already included code to create and train a new CNN called `net_with_bias`.
744
745 # Consider how a bias affects the equation for an unquantized layer:
746
747 #  $W * In + bias = Out$ 
748
749
750
751 # %%
752 ## Question 5.1:
753
754 # Suppose that we again quantized a biased layer with the same scaling factors we used in
previous questions:  $n_W$ ,  $n_{In}$ , and  $n_{Out}$ .

```



```

755 # What would we scale bias by in this case?
756 # Write an equation in your lab report to describe the output of the quantized layer with
    a bias.
757
758 # Create a new network with a bias on *fc3*.
759
760
761 # %%
762 class NetWithBias(nn.Module):
763     def __init__(self):
764         super(NetWithBias, self).__init__()
765         self.conv1 = nn.Conv2d(3, 64, 3, padding='same', bias=False)
766         self.bn1 = nn.BatchNorm2d(64, momentum= 0.1)
767         self.pool = nn.MaxPool2d(2, 2) # run after each conv (hence the 3x3 FC layer)    16
    x16
768         self.conv2 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
769         self.bn2 = nn.BatchNorm2d(64, momentum= 0.1)
770         self.conv3 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
771         self.bn3 = nn.BatchNorm2d(64, momentum= 0.1)
772         self.conv4 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
773         self.bn4 = nn.BatchNorm2d(64, momentum= 0.1)
774         self.conv5 = nn.Conv2d(64, 64, 3, padding='same', bias=False)
775         self.bn5 = nn.BatchNorm2d(64, momentum= 0.1)
776         self.conv6 = nn.Conv2d(64, 128, 3, padding=1, stride = 2, bias=False)    #8x8
777         self.bn6 = nn.BatchNorm2d(128, momentum= 0.1)
778         self.conv7 = nn.Conv2d(128, 128, 3, padding='same', bias=False)
779         self.bn7 = nn.BatchNorm2d(128, momentum= 0.1)
780         self.conv8 = nn.Conv2d(64, 128, 1, stride = 2, bias=False)
781         self.bn8 = nn.BatchNorm2d(128, momentum= 0.1)
782         self.pool2 = nn.AvgPool2d(2,2)
783         self.fc1 = nn.Linear(128 * 4 * 4, 128, bias=False)
784         self.fc2 = nn.Linear(128, 10, bias=True)
785         #定义前向传播,将tensor数据送入网络之后的计算图
786         def forward(self, x: torch.Tensor) -> torch.Tensor:
787             x = F.relu(self.conv1(x))
788             x = self.bn1(x)
789             x = self.pool(x)
790
791             temp = x
792             x = F.relu(self.conv2(x))
793             x = self.bn2(x)
794             x = F.relu(self.conv3(x))
795             x = self.bn3(x)
796             x = x + temp
797
798             temp = x
799             x = F.relu(self.conv4(x))
800             x = self.bn4(x)
801             x = F.relu(self.conv5(x))
802             x = self.bn5(x)
803             x = x + temp
804
805             temp = x
806             x = F.relu(self.conv6(x))
807             x = self.bn6(x)
808             x = F.relu(self.conv7(x))
809             x = self.bn7(x)
810             temp = F.relu(self.conv8(temp))
811             temp = self.bn8(temp)
812             x = x + temp
813
814             x = self.pool2(x)

```

```

815         x = x.view(-1, 128 * 4 * 4)
816         x = F.relu(self.fc1(x))
817         x = self.fc2(x)
818
819     return x
820
821
822 net_with_bias = NetWithBias().to(device)
823
824 # %%
825 # Train and test the model
826
827 # %%
828 train(net_with_bias, trainloader)
829
830 # %%
831 score = test(net_with_bias, testloader)
832 print('Accuracy of the network (with a bias) on the test images: {}'.format(score))
833
834 # %%
835 register_activation_profiling_hooks(net_with_bias)
836 test(net_with_bias, trainloader, max_samples=400)
837 net_with_bias.profile_activations = False
838
839 # %%
840 net_with_bias_with_quantized_weights = copy_model(net_with_bias)
841 quantize_layer_weights(net_with_bias_with_quantized_weights, bitwidth_W)
842
843 score = test(net_with_bias_with_quantized_weights, testloader)
844 print('Accuracy of the network on the test images after all the weights are quantized but
the bias isn\'t: {}'.format(score))
845
846 # %%
847 # Fill in the `quantized_bias` function in the `NetQuantizedWithBias` class.
848 # This function is meant to quantize the bias on the final layer of the CNN.
849 # Keep in mind that biases are typically quantized to 32-bits, so your bias values do not
all have to be between -128 and 127 (though 32-bits is a bit conservative).
850
851 # %%
852 # slightly clearer bias bounds (32b signed integer)
853 #bias量化后的最大值、最小值
854 MIN_32B_SINT = -(2**31)
855 MAX_32B_SINT = (2**31) - 1
856
857 # %%
858 class NetQuantizedWithBias(NetQuantized):
859     def __init__(self, net_with_weights_quantized: nn.Module):
860         super(NetQuantizedWithBias, self).__init__(net_with_weights_quantized, bitwidth_A)
861
862         preceding_scales_fc2 = [(layer.weight.scale, layer.output_scale) for layer in self
.children() if isinstance(layer, nn.Conv2d) or isinstance(layer, nn.Linear)][:-1]
863         #preceding_scales_fc1 = preceding_scales_fc2[0:4]
864         #print(preceding_scales_fc1)
865
866         #self.fc1.bias.data = NetQuantizedWithBias.quantized_bias(
867             #self.fc1.bias.data,
868             #self.fc1.weight.scale,
869             #self.input_scale,
870             #preceding_scales_fc1
871         #)
872         #print(self.fc1.bias.data)
873

```

```

874         self.fc2.bias.data = NetQuantizedWithBias.quantized_bias(
875             self.fc2.bias.data,
876             self.fc2.weight.scale,
877             self.input_scale,
878             preceding_scales_fc2
879         )
880         #print(self.fc2.bias.data)
881
882         #if (self.fc1.bias.data < MIN_32B_SINT).any() or (self.fc1.bias.data >
MAX_32B_SINT).any():
883             #raise Exception("Bias has values which are out of bounds for an 32-bit signed
integer")
884             #if (self.fc1.bias.data != self.fc1.bias.data.round()).any():
885                 #raise Exception("Bias has non-integer values")
886             if (self.fc2.bias.data < MIN_32B_SINT).any() or (self.fc2.bias.data > MAX_32B_SINT
).any():
887                 raise Exception("Bias has values which are out of bounds for an 32-bit signed
integer")
888                 if (self.fc2.bias.data != self.fc2.bias.data.round()).any():
889                     raise Exception("Bias has non-integer values")
890
891         @staticmethod
892         def quantized_bias(bias: torch.Tensor, n_w: float, n_initial_input: float, ns: List[
Tuple[float, float]]) -> torch.Tensor:
893             '''
894             Quantize the bias so that all values are integers between MIN_32B_SINT and
MAX_32B_SINT.
895
896             Parameters:
897             bias (Tensor): The floating point values of the bias
898             n_w (float): The scale by which the weights of this layer were multiplied
899             n_initial_input (float): The scale by which the initial input to the neural
network was multiplied
900             ns ((float, float)): A list of tuples, where each tuple represents the "weight
scale" and "output scale" (in that order) for every preceding layer
901             0
902             Returns:
903             Tensor: The bias in quantized form, where every value is an integer between
MIN_32B_SINT and MAX_32B_SINT.
904             The "dtype" will still be "float", but the values themselves should all be
integers.
905             '''
906
907             # ADD YOUR CODE HERE
908
909             #print(bias)
910             # 计算累积缩放因子: 初始输入缩放 * 所有前驱层的输出缩放
911
912             scale = n_initial_input
913             for weight_scale, output_scale in ns:
914                 scale *= output_scale
915                 scale *= weight_scale
916
917             # 总缩放因子 = 累积输入缩放 * 当前层权重缩放
918             scale = scale * n_w
919             print(scale)
920
921             # 量化偏置: bias * scale → 四舍五入 → 截断至32位整数范围
922             quan_bias = (bias * scale).round()
923             quan_bias = torch.clamp(quan_bias, min=MIN_32B_SINT, max=MAX_32B_SINT)
924             return quan_bias
925

```

```

926
927
928
929 # %%
930 net_quantized_with_bias = NetQuantizedWithBias(net_with_bias_with_quantized_weights)
931
932
933 # %%
934 ## Question 5.3:
935
936 # What is your accuracy before and after quantizing CNN with the bias?
937 # The accuracy change should ideally be negligible.
938
939 # %%
940 score = test(net_quantized_with_bias, testloader)
941 print('Accuracy of the network on the test images after all the weights and the bias are
quantized: {}'.format(score))

```

Listing 8: lab1_PTQ_resnet.ipynb

```

1 # %%
2 # 本次实验包括后量化的过程，主要在于学习如何对DNN的权重（weight）和激活层（activation）进
  行量化。
3
4 # %%
5 # 引用外部软件包
6 import torch # pytorch包
7 import torchvision # torchvision是pytorch官方提供的工具，包含很多常用数据集的封装等
8 import torchvision.transforms as transforms
9 import torch.nn as nn # nn 和下面的F均包含了主要的神经网络模块，包括卷积层、全连接层、池化
  层等等
10 import torch.nn.functional as F
11 import torch.optim as optim # 优化器
12
13 # %%
14 # 确保device为gpu，否则会花费过长时间。
15 if torch.cuda.is_available():
16     torch.cuda.set_device(0)
17     device = "cuda"
18 else:
19     device = torch.device('cpu')
20
21
22 # %%
23 # Load training and test data from the CIFAR10 dataset.
24 # torchvision是pytorch官方集成的工具库，包括学界常用的数据集和数据预处理方式。
25 transform = transforms.Compose(
26     [transforms.ToTensor(), # 将数据归一化，并转换成tensor的形式
27      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]) # 预处理数据的方式，
  normalize是将图像进行规则化。
28
29 trainset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset/cifar10', train=True,
30                                         download=True, transform=transform) #定义了训练
  集，path是训练数据所在地址。
31 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
32                                           shuffle=True, num_workers=2) #定义数据迭代器，
  batchsize为一次迭代训练，送进去图像的数量（可以尝试调整，看看会发生什么）。shuffle表示要不
  要打乱训练图像的排序。
33
34 testset = torchvision.datasets.CIFAR10(root='C:/Users/skyhappy/Desktop/学校的破事/作业/数
  字系统设计/lab1/dataset/cifar10', train=False,

```

```

35         download=True, transform=transform)
36     testloader = torch.utils.data.DataLoader(testset, batch_size=4,
37                                             shuffle=False, num_workers=2)
38
39     # %%
40     # Define a simple CNN that classifies CIFAR images.
41     # The network provided is similar to LeNet-5, and it has the following architecture:
42
43     # **Layer** |      **Type**      | **Input Shape** | **Output Shape** | **Activation**
44     # conv1 | Convolutional | 3x32x32 | 6x28x28 | ReLU
45     # pool1 | Max pool | 6x28x28 | 6x14x14 | None
46     # conv2 | Convolutional | 6x14x14 | 16x10x10 | ReLU
47     # pool2 | Max pool | 16x10x10 | 16x5x5 | None
48     # fc1 | Fully-connected | 400 | 120 | ReLU
49     # fc2 | Fully-connected | 120 | 84 | ReLU
50     # fc3 | Fully-connected | 84 | 10 | None
51
52     # None of the layers in the network have a bias associated with them.
53     # This makes them easier to quantize.
54     # Towards the end of this assignment, we will add biases to the final layer and quantize
55     # it as well.
56
57     # %%
58     # 重新定义conv函数与fc函数
59     import math
60     def TorchRound():
61         """
62         Apply STE to clamp function.
63         """
64         class identity_quant(torch.autograd.Function):
65             @staticmethod
66             def forward(ctx, input):
67                 out = torch.round(input)
68                 return out
69             @staticmethod
70             def backward(ctx, grad_output):
71                 return grad_output
72
73         return identity_quant().apply
74
75     def quant_max(tensor):
76         """
77         Returns the max value for symmetric quantization.
78         """
79         return torch.abs(tensor.detach()).max() + 1e-8 # 防止除0的情况出现
80
81     class quant_weight(nn.Module):
82         """
83         Quantization function for quantize weight with maximum.
84         """
85
86         def __init__(self, k_bits=8):
87             super(quant_weight, self).__init__()
88             self.k_bits = k_bits
89             self.qmax = 2. ** (k_bits - 1) - 1.
90             self.round = TorchRound()
91
92         def forward(self, input):
93             max_val = quant_max(input)
94             weight = input * self.qmax / max_val
95             q_weight = self.round(weight)

```

```

96         q_weight = q_weight * max_val / self.qmax # 通过这种方式引入量化误差, 从而实现QAT
97         #q_weight = self.round(q_weight)
98         return q_weight
99
100     class QuantConv2d(nn.Module):
101         """
102         A convolution layer with quantized weight.
103         """
104
105         def __init__(self, in_channels, out_channels, kernel_size, stride=1,
106             padding=0, dilation=1, groups=1, bias=False, k_bits=8): # 默认使用bias
107             #False的情况
108             super(QuantConv2d, self).__init__()
109             self.weight = nn.Parameter(torch.Tensor(out_channels, in_channels, kernel_size,
110                 kernel_size))
111             self.stride = stride
112             self.padding = padding
113             self.dilation = dilation
114             self.groups = groups
115             self.in_channels = in_channels
116             self.kernel_size = (kernel_size, kernel_size)
117             self.bias_flag = bias
118             if self.bias_flag:
119                 self.bias = nn.Parameter(torch.Tensor(out_channels))
120                 nn.init.constant_(self.bias)
121             else:
122                 self.register_parameter('bias', None)
123             self.k_bits = k_bits
124             self.quant_weight = quant_weight(k_bits = k_bits)
125             self.output = None
126             nn.init.kaiming_normal_(self.weight) # 初始化参数
127
128         def forward(self, input, order=None):
129             return F.conv2d(input, self.quant_weight(self.weight), self.bias, self.stride,
130                 self.padding, self.dilation, self.groups)
131
132     class QuantFC(nn.Module):
133         """
134         A Fully Connected layer with quantized weight.
135         """
136
137         def __init__(self, in_features, out_features, bias=False, k_bits=8): # 默认使用bias=
138             #False的情况
139             super(QuantFC, self).__init__()
140             self.weight = nn.Parameter(torch.Tensor(out_features, in_features))
141             self.bias_flag = bias
142             if self.bias_flag:
143                 self.bias = nn.Parameter(torch.Tensor(out_features))
144             else:
145                 self.register_parameter('bias', None)
146             self.k_bits = k_bits
147             self.quant_weight = quant_weight(k_bits = k_bits)
148             nn.init.kaiming_normal_(self.weight) # 初始化参数, 何恺明发明
149
150         def forward(self, input, order=None):
151             return F.linear(input, self.quant_weight(self.weight), self.bias)
152
153     # %%
154     class quant_act(nn.Module):
155         """

```

```

154     Quantization function for quantize activation with parameterized max scale.
155     """
156     def __init__(self, k_bits=8, decay=0.995):
157         super(quant_act, self).__init__()
158         self.k_bits = k_bits
159         self.qmax = 2. ** (self.k_bits - 1) - 1.
160         self.round = TorchRound()
161         self.decay = decay #平滑因子, 用于EMA, 能够更敏感地反映最新的数据变化
162         self.alpha = nn.Parameter(torch.Tensor(10)) #将激活值限制在 [-alpha, alpha] 范围
163         self.register_buffer('max_val', torch.ones(1))
164
165     def _ema(self, x): #通过指数移动平均 (EMA) 跟踪激活值的最大绝对值范围
166         act = x.reshape(x.shape[0], -1)
167         # print(act.shape)
168         max_val = torch.mean(torch.max(act, dim=1)[0])
169
170         self.max_val = (1.0-self.decay) * max_val + self.decay * self.max_val
171
172     def forward(self, x):
173         act = x
174         self._ema(x)
175         self.alpha.data = self.max_val.unsqueeze(0)
176         act = torch.max(torch.min(x, self.alpha), -self.alpha)
177
178         act = act * self.qmax / self.alpha
179         q_act = self.round(act)
180         q_act = q_act * self.alpha / self.qmax # 引入量化误差
181         #q_act = self.round(q_act)
182
183         return q_act
184
185 # %%
186 # 网络定义, conv2d是卷积层, linear是全连接层。forward下定义了计算图, 显示送进来的图像(x)
187 # 会怎么处理。
188 class Net(nn.Module):
189     # 网络结构:2层卷积层、1层最大值池化层、3层全连接层, 留意这些参数的内涵
190     def __init__(self):
191         super(Net, self).__init__()
192         self.conv1 = QuantConv2d(3, 64, 3, padding='same', bias=False)
193         self.pool = nn.MaxPool2d(2, 2) # run after each conv (hence the 3x3 FC layer) 16
194         self.conv2 = QuantConv2d(64, 64, 3, padding='same', bias=False)
195         self.conv3 = QuantConv2d(64, 64, 3, padding='same', bias=False)
196         self.conv4 = QuantConv2d(64, 64, 3, padding='same', bias=False)
197         self.conv5 = QuantConv2d(64, 64, 3, padding='same', bias=False)
198         self.conv6 = QuantConv2d(64, 128, 3, padding=1, stride = 2, bias=False) #8x8
199         self.conv7 = QuantConv2d(128, 128, 3, padding='same', bias=False)
200         self.conv8 = QuantConv2d(64, 128, 1, stride = 2, bias=False)
201         self.pool2 = nn.AvgPool2d(2,2)
202         self.fc1 = QuantFC(128 * 4 * 4, 128, bias=False)
203         self.fc2 = QuantFC(128, 10, bias=False)
204
205         self.quant_act1 = quant_act(k_bits=8) # activation for 8 bit
206         self.quant_act2 = quant_act(k_bits=8)
207         self.quant_act3 = quant_act(k_bits=8)
208         self.quant_act4 = quant_act(k_bits=8)
209         self.quant_act5 = quant_act(k_bits=8)
210         self.quant_act6 = quant_act(k_bits=8)
211         self.quant_act7 = quant_act(k_bits=8)
212         self.quant_act8 = quant_act(k_bits=8)
213         self.quant_act9 = quant_act(k_bits=8)

```

```

213
214     #定义前向传播,将tensor数据送入网络之后的计算图
215     def forward(self, x: torch.Tensor) -> torch.Tensor:
216         x = F.relu(self.conv1(x))
217         x = self.pool(x)
218         x = self.quant_act1(x)
219
220         temp = x
221         x = F.relu(self.conv2(x))
222         temp = self.quant_act2(temp)
223         x = self.quant_act2(x)
224         x = F.relu(self.conv3(x))
225         temp = self.quant_act3(temp)
226         x = self.quant_act3(x)
227         x = x + temp
228
229
230         temp = x
231         x = F.relu(self.conv4(x))
232         temp = self.quant_act4(temp)
233         x = self.quant_act4(x)
234         x = F.relu(self.conv5(x))
235         temp = self.quant_act5(temp)
236         x = self.quant_act5(x)
237         x = x + temp
238
239
240         temp = x
241         x = F.relu(self.conv6(x))
242         temp = self.quant_act6(temp)
243         x = self.quant_act6(x)
244         x = F.relu(self.conv7(x))
245         temp = self.quant_act7(temp)
246         x = self.quant_act7(x)
247         temp = F.relu(self.conv8(temp))
248         temp = self.quant_act8(temp)
249         x = self.quant_act8(x)
250         x = x + temp
251
252
253         x = self.pool2(x)
254
255         x = x.view(-1, 128 * 4 * 4)
256         x = F.relu(self.fc1(x))
257         x = self.quant_act9(x)
258         x = self.fc2(x)
259
260         return x
261
262
263     net = Net().to(device)
264     # 到这里,我们定义好了网络和数据集的使用方式
265
266     # %%
267     from torch.utils.data import DataLoader
268     #将训练集的dataloader送入网络进行训练,每次读取batchsize数量的数据
269     def train(model: nn.Module, dataloader: DataLoader):
270         criterion = nn.CrossEntropyLoss() # loss function (交叉熵损失函数)
271         optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) #optimizer, 使用SGD
272         #优化器训练,第一个参数表示model的所有参数都会参与训练,学习率为0.001,动量为0.9
273
274         for epoch in range(2): # loop over the dataset multiple times。epoch指遍历所有训练集
275             #图像的个数。这里遍历所有训练图像三次
276
277             running_loss = 0.0

```



```

273         for i, data in enumerate(dataloader, 0): # dataloader是数据迭代器，从这里每次读出
batchsize数量的数据。
274             # get the inputs; data is a list of [inputs, labels]
275             inputs, labels = data
276
277             inputs = inputs.to(device)
278             labels = labels.to(device)
279
280             # zero the parameter gradients
281             optimizer.zero_grad()
282
283             # forward + backward + optimize
284             outputs = model(inputs) # 将数据送进网络，得到结果
285             loss = criterion(outputs, labels) # 计算结果和标签的交叉熵损失函数
286             loss.backward() # 基于上述损失函数求解梯度
287             optimizer.step() # 更新参数
288
289             # print statistics
290             running_loss += loss.item() # 将loss的值取出来，记录其变化
291             if i % 2000 == 1999: # print every 2000 mini-batches
292                 print('%d, %5d loss: %.3f' %
293                       (epoch + 1, i + 1, running_loss / 2000))
294                 running_loss = 0.0
295
296             print('Finished Training')
297
298     def test(model: nn.Module, dataloader: DataLoader, max_samples=None) -> float:
299         correct = 0
300         total = 0
301         n_inferences = 0
302
303         with torch.no_grad(): # 测试时不用求解梯度，因此可以设置不计算梯度的模式
304             for data in dataloader:
305                 images, labels = data
306
307                 images = images.to(device)
308                 labels = labels.to(device)
309
310                 outputs = model(images) # get 1 batch worth of image predictions (i.e. 4
predictions of 10 each)
311                 other, predicted = torch.max(outputs.data, 1) # other == values, predicted ==
indicies
312                 total += labels.size(0)
313                 correct += (predicted == labels).sum().item()
314
315                 if max_samples:
316                     n_inferences += images.shape[0]
317                     if n_inferences > max_samples:
318                         break
319
320             return 100 * correct / total
321
322     # %%
323     train(net, trainloader) # 进行训练，将网络定义和数据迭代器传给train函数。
324
325     # %%
326     # Now that the CNN has been trained, let's test it on our test dataset.
327     score = test(net, testloader)
328     print('Accuracy of the network on the test images: {}'.format(score))
329
330     # %%
331     # Define a convenience function which we use to copy CNN's.

```

```

332 from copy import deepcopy
333
334 def copy_model(model: nn.Module) -> nn.Module:
335     result = deepcopy(model)
336
337     # Copy over the extra metadata we've collected which copy.deepcopy doesn't capture
338     if hasattr(model, 'input_activations'):
339         result.input_activations = deepcopy(model.input_activations)
340
341     for result_layer, original_layer in zip(result.children(), model.children()):
342         if isinstance(result_layer, nn.Conv2d) or isinstance(result_layer, nn.Linear):
343             if hasattr(original_layer.weight, 'scale'):
344                 result_layer.weight.scale = deepcopy(original_layer.weight.scale)
345             if hasattr(original_layer, 'activations'):
346                 result_layer.activations = deepcopy(original_layer.activations)
347             if hasattr(original_layer, 'output_scale'):
348                 result_layer.output_scale = deepcopy(original_layer.output_scale)
349
350     return result
351
352 # %%
353 # Visualize Weights
354
355 # %%
356 import matplotlib.pyplot as plt
357 import numpy as np
358
359 name = ['conv1', 'conv2', 'conv3', 'conv4', 'conv5', 'conv6', 'conv7', 'conv8', 'fc1', '
360         fc2']
361 n = 0
362
363 for layer in net.children(): # 遍历类的成员
364     if isinstance(layer, QuantConv2d) or isinstance(layer, QuantFC):
365         weight = layer.weight.data.cpu().view(-1).numpy()
366         print(name[n] + ": the range is [{:.5f}, {:.5f}].format(weight.min(), weight.max
367               ()))
368
369         # find 3-sigma range (u-3s, u+3s), 该范围能囊括99.73%参数
370         sigma3 = int(np.size(weight)*0.0027)
371         sigma3_value = np.abs(weight)[np.argsort(np.abs(weight), -sigma3)[-sigma3]]
372         print(name[n] + ": the 3-sigma range is [{:.5f}, {:.5f}].format(max(-sigma3_value
373               , weight.min()), min(sigma3_value, weight.max()))
374
375         plt.hist(weight, bins=50, facecolor="blue", edgecolor="black", alpha=0.7) # 画直方
376         图
377
378         plt.xlabel("range")
379         plt.ylabel("frequency")
380         plt.title(name[n])
381         plt.show()
382         n += 1
383
384 # You can get a flattened vector of the weights of fc1 like this:
385 # fc1_weights = net.fc1.weight.data.cpu().view(-1)
386
387 # %%
388 score = test(net, testloader)
389 print('Accuracy of the quantized network on the test images: {}'.format(score))

```

Listing 9: lab1_QAT_ResNet.ipynb